

A Unified Propagation Framework for Reliability, Scheduling and Flow Analysis in Directed Acyclic Process Networks

by

[Temitope Ohiani]

A thesis presented in partial fulfilment of the requirements for the
degree of

Doctor of Philosophy

Department of Civil Engineering
University of Strathclyde

[Month Year]

Supervised by: [Edoardo Pattelli]

Co-supervised by: [Caroline Pyke]

Declaration of Authenticity and Author's Rights

This thesis is the result of the author's original research. It has been composed by the author and has not been previously submitted for examination which has led to the award of a degree.

The copyright of this thesis belongs to the author under the terms of the United Kingdom Copyright Acts as qualified by University of Strathclyde Regulation 3.50. Due acknowledgement must always be made of the use of any material contained in, or derived from, this thesis.

Signed: _____

Date: _____

Abstract

Complex processes and infrastructures require careful planning and monitoring. Be it at the design stage or during normal operations to eventual decommissioning, civil infrastructure systems require specialized modelling and analysis tools across their entire lifecycle. Through all of this, design decisions propagate through the system in ways that are difficult to predict without a unified analytical view of the system. Operators, designers, and planners working on the same process at the same time may be asking fundamentally different questions of it. In these systems, performance objectives and success criteria demand different combinations of tools.

This usually leads to separate representations of the same system are maintained in parallel, with little overlap, shaped by the specific metadata each use case requires. As systems scale in size and complexity, the diversity of these analysis needs continues to grow while fragmentation further increases.

Rarely does a framework exist that uses a single shared model of the network to support multiple analysis objectives. This work presents such a framework: an integrated multi-algorithm system for performance analysis of flow-oriented processes on directed acyclic graph (DAG) networks, built around a single shared network model that supports multiple analysis objectives without redefining the underlying network.

The Information Propagation Framework is a collection of exact propagation algorithms: probability propagation with exact belief propagation over DAGs including structural handling of convergent subgraphs via diamond decomposition; a generalised Critical Path Method that supports configurable operator combinations for scheduling and cost analysis; and a capacity analysis pipeline covering maximum flow, sensitivity analysis, failure impact, and structural vulnerability.

In this work, the propagation algorithms are parametric over three possible uncertainty representations (deterministic values, interval bounds, and probability boxes) to allow users the flexibility to run any of the available analysis endpoints under different assumptions about input uncertainty. Multiple case studies are presented to demonstrate the efficiency and accuracy of all implemented algorithms across networks of varying size and complexity.

The framework is implemented as an open-source Julia package with a code-base designed to be adapted or extended for specific use cases. To reduce the barrier to adoption for users less familiar with Julia or scientific programming, the framework is also delivered as a graphical web application. The front-end operates on user-owned local data and serves as a practical profiling tool for evaluating design robustness, identifying resource bottlenecks, assessing scheduling risk, and tracing how design decisions propagate through downstream constraints across different scenarios and metrics.

Acknowledgements

[Your acknowledgements text here. Typically includes thanks to: - Supervisors
- Funding bodies (EPSRC, industrial sponsors, etc.) - Industry collaborators
at Sellafield/NDA - Research group colleagues - Family and friends - Technical
support staff]

Contents

Declaration of Authenticity and Author's Rights	2
Abstract	3
Acknowledgements	5
Nomenclature	14
1 Introduction	15
2 Literature Review	16
3 Complex Processes as Directed Acyclic Graphs	17
3.1 Introduction	18
3.2 From Process to Graph	18
3.3 Node Roles	19
3.4 The Formal Object	20
3.5 Information on the Network	21
3.6 Assumptions and Limits	22
3.7 Three Questions	22
4 Input Processing Module	23
4.1 Introduction	24
4.2 File Parsers	25
4.2.1 Network File Parser	25
4.2.2 MetaData Parser	26
4.3 IPF's Graph Object	28
4.3.1 Topological Sorting in DAGs	28
4.3.2 Extended Network Properties	29
4.3.3 The Unified Graph Object	31
4.3.4 Why not Julia's Native Graph Object?	31

5	Sub-graph Decomposition: Diamond Processing Module	35
5.1	Introduction	36
5.1.1	Decomposition in Complex Networks	36
5.2	Preliminaries and Definitions	37
5.3	A Configurable Target Subgraph	38
5.4	The Identification Algorithm	39
5.4.1	Conditioning only where correlation exists	39
5.4.2	Independence partitioning	40
5.4.3	Fork selection	40
5.4.4	The recursive loop	40
5.4.5	Edge construction and isolation	41
5.4.6	Context-aware identity	41
5.4.7	The exclusion predicate <code>is_det</code>	42
5.5	Structural Correctness	43
5.6	Representation and Reuse	46
5.6.1	Diamonds as graph objects	46
5.6.2	Deduplication by hash-consing	47
5.7	Worked Examples	48
5.7.1	The atomic diamond	48
5.7.2	A nested decomposition	48
5.8	Summary	51
6	Information As Probabilities: Probability Propagation Toolkit	52
7	Information As Resources: Capacity Flow Toolkit	53
7.1	Introduction	54
7.2	Mathematical Model and Network Flow Semantics	55
7.3	Available Modules	60
7.4	Base Flow Module	61
7.4.1	Max-Flow Min-Cut Solvers	61
7.4.2	Push-Relabel Solver	63
7.5	Network Flow Diagnostics	63
7.5.1	Min-Cut Utility Module	63
7.5.2	Structure Utility Module	65
7.5.3	Flow Sensitivity Module	66
7.5.4	Failure Impact Module	68
7.5.5	Parametric Threshold Module	69
7.6	Exactness, Validation, and Scope	70
7.6.1	Post-solve validation	70
7.6.2	Scope of results	71

7.6.3	Computational Complexity	71
7.7	Network Flow Output Structure	72
7.8	Case Study: Synthetic Benchmark Network	75
7.8.1	Benchmark Network Design	75
7.8.2	Comparative Results (Config A vs Config B)	76
Appendices		87
8	Information As Schedule and Cost: Critical Path Toolkit	89
8.1	Introduction	90
8.2	Scheduling Analysis on Networks	90
8.3	Analysis Modes	91
8.4	The Two Kernels	92
8.5	Interval Analysis	93
8.6	Exact Interval Floats: the Domination Split	94
8.7	Case Studies	97
8.7.1	One network, four questions	97
8.7.2	The most reliable route	99
8.7.3	The boundary	101
8.8	Genericity	102
8.9	Summary	103
9	Package Implementation in Julia	104
10	The No-Code Web Interface	105
11	An End-to-End Framework Application: An Integrated Case Study	106

List of Figures

3.1	A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.	19
3.2	Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.	20
5.1	Edge isolation at a nested join	42
5.2	The atomic diamond.	48
5.3	Frame 1. The network, with forks outlined in bold and joins shaded.	49
5.4	Frame 2. The maximal diamond at join 8, with fork 1 conditioned (filled) and the span covering the whole network.	49
5.5	Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 selected, the inner diamond in bold, and edge (1,3) cut.	50
5.6	Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.	51
7.1	Minimal single-source single-sink capacitated DAG network.	55
7.2	Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.	57
7.3	Node-splitting transformation.	58
7.4	Runtime workflow for the flow-capacity toolkit with available modules	60
7.5	Single-edge sensitivity in Config A and cross-configuration rank comparison (A vs B).	79
7.6	Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.	82

7.7	Sensitivity ranking for non-zero ΔF^* values in Configurations A and B.	83
7.8	Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.	83
7.9	Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.	84
8.1	The running example. Node durations 1, 5, 2, 1; LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4. . .	93
8.2	Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.	95
8.3	The N pattern that places u in a bypass set: u reaches v and also reaches a sink around it. Series-parallel orders contain no such pattern.	97
8.4	The water network read for both extremes. The bold double-bordered chain sets the minimum release timeline of 45 units. The shaded stages and dashed edges form the tied-fastest route system, every path through which completes in 15 units. The two readings share no stage.	98
8.5	Exact float bounds on the water network with every duration at plus or minus 20 percent. Black bars are the nine possibly-critical stages, whose lower bounds reach zero. Grey bars are three of the 23 stages whose floats stay provably positive everywhere in the uncertainty box.	100
8.6	Best-route success across the 323 drone destinations. Left of the dashed line at 0.5 lie the 104 destinations that no single route serves reliably.	102

List of Tables

3.1	Notation established by the network model.	21
4.1	Input Processing Contracts by Analysis Class	28
4.2	Components of the Unified Graph Object	32
5.1	Field correspondence between the unified graph object of Chapter 4 and <code>DiamondComputationData</code>	47
7.1	Max-flow solver comparison.	61
7.2	Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.	72
7.3	Unified output schema for network flow capacity analysis across all modules.	74
7.4	Configuration comparison summary.	76
7.5	Node-upgrade sweep (Configurations A and B give identical res- ults): minimum single-node capacity increase required to restore <code>sink_1</code> service under the node-capacitated solve.	78
7.6	Parametric threshold and marginal range comparison for key edges.	81
7.7	Min-cut enumeration for Config A.	81
8	Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).	88
9	Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).	88
8.1	The shipped modes. Every row carries a defined backward se- mantics; the margin column names what the backward pass reports.	91
8.2	LongestPath on the water network: completion 45 along the crit- ical chain, with the near-critical pair at slack 1.5.	98
8.3	Accumulation on the water network under a budget of 528 units at outlet 27 (current total 480). Allowance is headroom divided by path multiplicity.	99

8.4	The five best-served destinations by single-route success. At the other end of the scale, 104 of the 323 destinations fall below 0.5 on even their best route.	101
-----	--	-----

List of Algorithms

1	FindIterationSetsWithClosure	30
2	Recursive diamond identification (skeleton)	39
3	Ford-Fulkerson Min-Cut Max-Flow	62
4	Push-Relabel Min-Cut Max-Flow	63

Nomenclature

Abbreviations

IPA	Information Propagation Algorithm
IPA	Information Propagation Framework
DAG	Directed Acyclic Graph
MC-MC	Min-Cut Max-Flow
DFS	Depth First Search
BFS	Breath First Search
BP	Belief Propagation
CPM	Critical Path Method
MC	Monte Carlo
NDA	Nuclear Decommissioning Authority
p-box	Probability Box

Chapter 1

Introduction

Chapter 2

Literature Review

Chapter 3

Complex Processes as Directed Acyclic Graphs

3.1 Introduction

The systems this thesis serves are flow-oriented. Water is abstracted, treated and distributed. Power is generated, transformed and delivered. Materials move through processing stages towards demand points, and project activities complete in an order their dependencies dictate. In each case the behaviour of the whole is organised by a directed movement of something, supply, product, work or information, through a network of components that pass it on.

The framework rests on a single model of such systems, defined in this chapter and consumed unchanged by every module that follows. The model fixes what a system becomes when it enters the framework, what its components and their relationships mean, what a value attached to a component can stand for, and which assumptions the later analyses inherit. It closes with the three questions the model makes precise, which the three analysis toolkits of Chapters 6, 7 and 8 answer.

3.2 From Process to Graph

A system enters the model through two identifications. Components become nodes, and dependencies become directed edges. A component is whatever unit of the system can meaningfully carry a value of its own, a pump, a treatment stage, a substation, a task, and a dependency is the fact that one component feeds, supplies or precedes another. Direction is not an abstraction imposed on the system but a property read off it, because in a flow-oriented process the relation between two connected components is already asymmetric. Water leaves the intake and arrives at the pump, and the pump does not supply the intake.

The model further requires that the directed edges form no cycle, and this is the one modelling decision that needs defending rather than reading off. For a large class of the target systems acyclicity is simply true. Treatment trains, supply chains, assembly processes and project schedules move material or work strictly forward, and a dependency cycle in a schedule would be a planning error rather than a modelling choice. For other systems it is an orientation of a richer reality. A distribution pipe can carry water in either direction in principle, yet a network operating around its design flows has a dominant direction on every pipe, and orienting each edge by that dominant flow yields a directed acyclic snapshot of the operating regime. What the orientation gives up is the analysis of regimes far from the snapshot, flow reversals under unusual demand, recirculation loops, feedback control, and the model excludes these deliberately rather than approximating them. Figure 3.1 shows the mapping on a small treatment process,

whose deliberate duplication of the treatment stage already displays the structural phenomenon that Section 3.3 names.

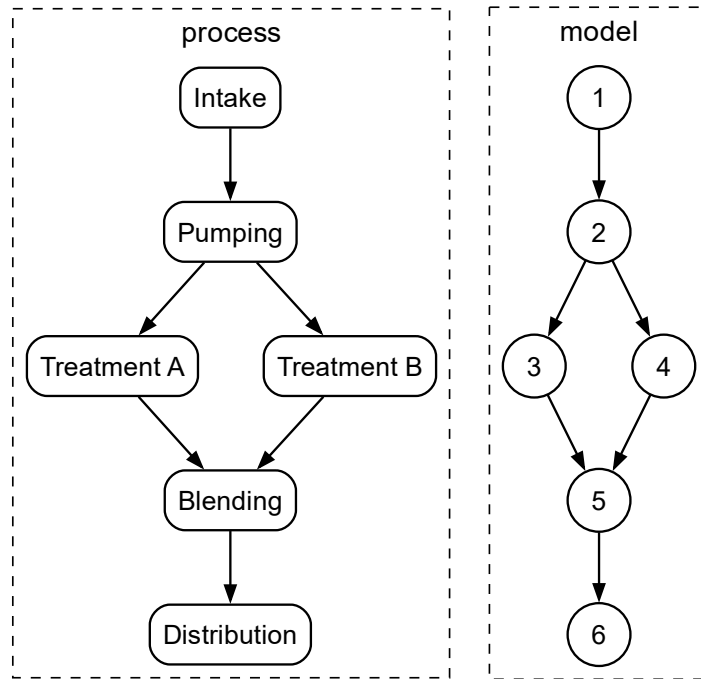


Figure 3.1: A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.

3.3 Node Roles

Position in the network gives every node a role, and each role has a physical reading. A node with no incoming edge is a source, the point where supply, material or work enters the system, an intake, a generator, the start of a project. A node with no outgoing edge is a sink, where the system delivers, a demand point, an outlet, a completed handover. A node that feeds two or more downstream components is a fork, and forks are how systems distribute and, above all, how they build in redundancy, because providing two routes to the same destination begins with one component supplying two. A node fed by two or more components is a join, where distributed flows gather, where assembled parts meet, and where redundant routes come back together.

The last of those readings deserves emphasis, because it is a fact about engineered systems rather than about graphs. Robust systems duplicate deliberately,

and duplicated routes must reconverge at the destination they protect, so a well-designed network is dense with paths that separate at a fork and meet again at a join while sharing the components upstream of the fork. Whether that sharing matters depends on the question being asked, and the chapters that ask the questions will return to it. Chapter 5 makes the pattern itself precise and Chapter 6 shows why, for one of the three readings of the model, ignoring it produces wrong answers. Here it is enough that reconvergence is a designed feature of the systems being modelled, visible already in Figure 3.2, not an accident of drawing.

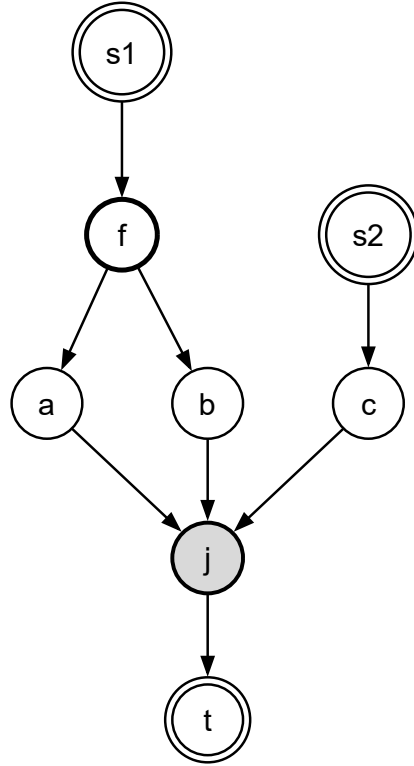


Figure 3.2: Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.

3.4 The Formal Object

Definition 3.1 (Network model). *A system is modelled as a finite directed acyclic graph $G = (V, E)$ with node set V and edge set $E \subseteq V \times V$. The sources $S \subseteq V$ are the nodes with no incoming edge and the sinks $T \subseteq V$ the nodes with no outgoing edge. For a node v , $\text{Pa}(v)$ denotes its parents, the nodes with an edge into v , and $\text{Ch}(v)$ its children.*

Definition 3.2 (Ancestry). *A node u is an ancestor of v when a directed path leads from u to v , and a descendant when one leads from v to u . The sets $\text{anc}(v)$ and $\text{desc}(v)$ collect them, and throughout the framework both are taken self-inclusively, so that $v \in \text{anc}(v)$ and $v \in \text{desc}(v)$.*

Definition 3.3 (Forks and joins). *The forks of G are $F = \{v : |\text{Ch}(v)| \geq 2\}$ and the joins are $J = \{v : |\text{Pa}(v)| \geq 2\}$.*

These definitions are the shared vocabulary of the thesis. The chapters that follow reference them rather than restate them, and Table 3.1 collects the notation in one place.

Table 3.1: Notation established by the network model.

Symbol	Meaning
$G = (V, E)$	the system as a finite DAG
S, T	sources (no incoming edge), sinks (no outgoing edge)
$\text{Pa}(v), \text{Ch}(v)$	parents and children of v
$\text{anc}(v), \text{desc}(v)$	ancestors and descendants, self-inclusive
F, J	fork nodes (fan-out ≥ 2), join nodes (fan-in ≥ 2)

3.5 Information on the Network

The model separates what the network is from what is asked of it. The topology of Definition 3.1 is fixed once per system, and an analysis attaches values to its nodes and edges whose meaning depends on the question. Under the probabilistic reading a node's value is the probability that the component operates and an edge's value the probability that the handover between its endpoints succeeds, so the network describes what survives failure. Under the schedule and cost reading the values are durations or costs, of the component's own processing and of the transfer along each dependency, so the network describes when things complete and what they accumulate. Under the capacity reading the values are throughput limits, and the network describes how much can be carried. Three readings of one topology are three different analyses, and keeping the topology common across them is what allows a single system model to answer questions that are usually asked of three separate models [1].

How well a value is known is a further axis, independent of what it means. A deterministic value asserts full knowledge, and is the right form for design constants and well-measured components. An interval asserts only bounds, and is the honest form when data is sparse, when a manufacturer quotes a tolerance,

or when experts will commit to a range but not a number. A probability box bounds a distribution rather than a value, and carries partial statistical knowledge, measurements enough to constrain a curve without fixing it. The framework propagates all three forms through the same analyses, so the choice among them reflects the state of knowledge about the system rather than the limits of the tool.

3.6 Assumptions and Limits

The model buys its tractability with four assumptions, and each is a limit on what the answers mean. Component behaviours are independent, so a common cause that fails several components together is not represented and must be modelled explicitly as a shared upstream node where it matters. Under the probabilistic reading a component is binary, operational or failed, and degraded intermediate states are outside the model. The topology is static over the analysis horizon, so reconfiguration during operation is a sequence of separate analyses rather than one. And the acyclicity of Section 3.2 prices out feedback and reversal, which is a property of the operating snapshot being analysed rather than of the physical assets. None of these assumptions is hidden in an algorithm, and every result in the later chapters is conditional on exactly this list.

3.7 Three Questions

The model makes three questions precise, and the thesis answers them in turn. Under the probabilistic reading the question is whether information survives the network, with what probability a node is reached from the sources despite component failures, and it is answered by the Probability Propagation Toolkit of Chapter 6. Under the capacity reading the question is how much the network can deliver, where the limits bind and what relieving them buys, answered by the Capacity Flow Toolkit of Chapter 7. Under the schedule and cost reading the question is when and at what accumulation the network completes, which dependencies set that outcome and which have room to move, answered by the Critical Path Toolkit of Chapter 8.

Before any of the three can be asked, the model itself must be built from the heterogeneous descriptions in which real systems arrive, which is the work of Chapter 4, and for the probabilistic question the reconvergent structure of Section 3.3 must be identified and organised, which is the work of Chapter 5. The two are preprocessing in the strict sense, performed once per system, and everything after them is a reading of the object this chapter has defined.

Chapter 4

Input Processing Module

4.1 Introduction

When it comes to adopting a tool like IPF, the first consideration will always be about dataset format and model compatibility. Understanding complex civil infrastructures often requires measuring multiple different metrics and objectives. The same system or process might get a different type of treatment across different teams within the same project.

Attribute-level information often gets split across silos of project objectives. Each silo usually only maintains the network parameters and behaviours specific to the operations concerned. Within the same infrastructure, one workflow may track fault/health signals while another tracks project related costs. For example, a fault detection objective is unlikely to track cost impact metrics or environmental load.

This could be because of scope restriction or lack of access to that additional data. Beyond scope of analysis, the stage of the system's lifecycle might affect the kind of data available, as planning and design needs evolve alongside the tools used as the system matures.

The actual tools used across infrastructure operations often implement different schemas and export conventions which may introduce more formatting inconsistencies. Sometimes the dataset has been purpose built to fit a specific tool or standard. Despite this, from a purely structural perspective, the network model is simply a collection of edges describing some topology, process, or flow of activities. Regardless of variation in source format, any DAG-based system model can generally be reduced to a node list and an edge list, in which nodes represent point features such as sources, junctions, assets, or demand points, and edges represent the relationships between these assets. This is consistent with widely used infrastructure data formats, such as GraphML, EPANET .inp, and MAT-POWER .m, which all reduce to node and edge records beneath their respective schemas.

However, when system characteristics and behaviours come into play, the structure is not so clear-cut. The relationships may carry information about the system's behaviour or analysis touchpoints such as resource capacity. This work takes a split perspective of network structure vs analysis inputs as different layers of the same network model. The initial dataset may combine these, but the goal of this one-time preprocessing step is to parse different formats into what the framework needs irrespective of source schema.

The Input Processing Module is the interface between heterogeneous raw inputs that describe the system's characteristics and the standardised network model used throughout the framework. This module is therefore necessarily the

sole mandatory entry point, without which no downstream algorithm can be validly invoked. This is implemented as a two-part transformation: parsing different raw input file format, and constructing the standardised graph object the framework operates on.

4.2 File Parsers

4.2.1 Network File Parser

Nodes in the physical system typically have descriptive identifiers that reflect their role in the network (Pump_A, Substation_North). However, integers are more efficient in terms of memory footprint while allowing faster indexing during the repeated lookups and set operations performed throughout the framework.

The network parser exposes a reader function that expects a .Edges or .CSV network structure file containing either an integer-keyed edge list or an adjacency matrix. Although these integers are not required to be contiguous, the user is responsible for mapping their domain-specific identifiers to unique integers before parsing the network into the framework.

The module uses a staged detection flow to automatically detect file types when called. Given file path, the reader function always prefers detection via file extension. This allows for .edges files to immediately trigger the edge list reader. On the other hand, a .csv file is ambiguous since it could contain either an adjacency matrix or a column-formatted edge list. In this case, the next detection stage is lightweight content sniffing that scans headers for common source-destination keywords that indicate an edge list.

If no matching header with expected descriptors is present, the next stage scans the file content for the integer-delimiter-integer pattern that usually indicates an edge list. The last edge list indicator checks the shape of the table itself where a two-column Nx2 shape is treated as an edge list, otherwise the file defaults to adjacency matrix parsing.

When an adjacency matrix is detected instead, the reader first streams the table row by row to build an effective edge list. As the edges are streamed in from either file type, self-loops are rejected immediately and cycle-forming edges are detected during ingestion, since both violate the acyclicity expectation of the target DAG model.

To keep the ingestion process linear in the number of edges, the reader function builds both the incoming and outgoing index directions in place in order to avoid a second pass that would otherwise be needed to construct the reverse adjacency. During the stream, for every valid edge discovered, both nodes and

their directional relationships are recorded in the same pass. Once the stream completes, the parser returns the framework’s minimal graph object.

4.2.2 MetaData Parser

Given a validated base graph object, the framework supports both node-level and edge-level information depending on the algorithm invoked. The analysis-specific input files are expected to hold the network parameters that drive each downstream algorithm. Three classes of propagation characteristics are supported: critical path, flow capacity, and probabilistic information. The sources of this information are usually decoupled from network modelling. Instead they tend to be observed externally and managed separately (such as manufacturer specs and historic data).

In large networks, embedding these behavioral properties into the network structure leads to a bloated file that can be harder to maintain. For multi-metric evaluation on the same network, the user would be forced to maintain duplicate topology files per analysis variant even when the network structure does not change.

To this end, regardless of analysis type, all algorithm inputs are required as .JSON files. JSON is a human-readable, platform-agnostic format that maps naturally to the key-value shape for describing component attributes. Where flat formats like CSV have a rigid tabular shape that makes them a good fit for describing the graph, JSON’s flexibility allows for a more expressive representation of system characteristics. Intervals, p-boxes, and other nested data types can be easily described by data owners without the need to invent column conventions or force fit flattened values on inherently nested properties. Since it also requires no specialist tools to construct or edit, as long as the user understands the node and edges mapping, then no special understanding of graph theory or modelling tools is required to contribute values.

Schema Conventions and Type Dispatch

Although metadata labels are useful for documentation and reporting, the parser’s execution is driven by parseable values, key shape, and algebraic compatibility with the specified analysis type. To determine the appropriate parser for different algorithm’s inputs, any JSON file must declare a `data_type` (scalar, interval, p-box) as well as an `analysis_type` (flow capacity, critical path, probability).

Irrespective of the analysis types, the rest of the schema contract is simple: values must be mappable to valid node identifiers and edge identifiers from the parsed DAG, and values must be parseable into the declared type domain. This

convention aims to ensure that uncertainty deserialisation is concentrated in one place rather than spread across multiple readers, while keeping the JSON interface flexible to different naming conventions.

Critical Path inputs describe node and edge weights in terms of critical-path style propagation. Since the framework is deliberately unit-agnostic for this analysis class, the weights may represent durations, costs, risk scores, environmental impact, or any other accumulative quantity, as long as the interpretation is consistent within a single run. The parser does not discriminate by unit-specific labels such as hours or risk points; it is instead driven by runtime operator functions. In addition to the node-value and edge-value maps, the input expects three operator functions to be specified: a combination function, a propagation function, and a node function, which are discussed further in the critical path methodology chapter. These operators are what define the analysis semantics at runtime and may be left unspecified, in which case the framework applies classical critical-path defaults.

Flow Capacity inputs describe throughput constraints across the network. Edge and node capacities may be supplied in the same JSON file or split across separate files; the parser infers the value mapping based on key structure. Edge capacities are the main requirement, with optional node capacities. Unlike the other two analysis classes, capacity in this work is not uncertainty-polymorphic and only handles fixed values. To model unconstrained capacities, the metadata reader supports standard infinity tokens including `Inf`, `+Inf`, `Infinity`, and ∞ . Given this, numeric soundness of capacity inputs is validated upfront, with non-negative and non-NaN constraints enforced at parse time and any malformed or invalid input rejected early.

Probabilistic Inputs describe uncertainty attached to nodes and edges, typically for reliability and probability propagation. Node priors capture component-level uncertainty and edge probabilities capture link-level transmission or failure likelihoods. All probability values, whether scalar, interval, or p-box, must remain within $[0,1]$; the framework rejects values outside this range at parse time. This is the most expressive of the three analysis classes, supporting all three data types. When a non-scalar data type is specified, the values contained must match the expected shape. For instance, an interval is expected as either a delimited tuple or nested properties keyed by upper and lower bounds. P-boxes, in contrast, must follow a tagged serialization contract and declare a `construction_type`. The p-box representation, including the construction type, is expected to mirror the constructor families supported by `ProbabilityBoundsAnalysis.jl`.

Table 4.1: Input Processing Contracts by Analysis Class

Layer / Class	Contract Shape	Value Domain	Validation / Dispatch
Network topology	Edge list or adjacency matrix over integer node IDs	Integer node IDs; directed edges	Format auto-detected; self-loops and cycles rejected at parse
Probability propagation	Node map and edge map keyed to DAG identifiers	Float64, Interval, pbox	Values within $[0,1]$; key shape separates node and edge maps; <code>data_type</code> drives deserialisation
Flow capacity	Edge capacity map (required), node capacity map (optional)	Float64 only	Non-negative, non-NaN enforced at parse; Inf tokens accepted (Inf, +Inf, ∞)
Critical path	Node-value and edge-value maps	Float64, Interval	Unit-agnostic; semantics set by runtime operators (combination, propagation, node); classical defaults applied if unspecified

4.3 IPF's Graph Object

4.3.1 Topological Sorting in DAGs

For the directed acyclic graph $G = (V, E)$, the topological sort T is a linear ordering of nodes where every node appears exactly once, and for every edge $u \rightarrow v$, node u must appear before v . This order can equivalently be expressed in layered sets such that each layer groups nodes that are at the same depth from the source nodes and do not have dependencies within the group.

Theorem 4.1 (IPF's Convergence Guarantee). *Given a DAG $G = (V, E)$, an information propagation algorithm will always converge in exactly $|L|$ iterations, where L is the topologically sorted layers of G .*

Proof. We prove by induction on the number of nodes $|V| = n$ that for any DAG $G = (V, E)$ there exists a valid topological sort T and an equivalent layered topological sort $L' = \{L_1, L_2, \dots, L_k\}$.

- *Base case:* If G contains only one node, then the topological sorting T is simply the node itself, and L contains a single set containing the node. Thus, $|L| = 1$ and the single layer contains only this source node.
- *Inductive Assumption:* Assume that for any DAG G with $n - 1$ nodes, there exists a valid topological sort T of G and a layered sort $L = \{L_1, \dots, L_{k-1}\}$.

- *Inductive Step:*
 - First subset: Since G is a DAG, at least one source node must exist. We group all the source nodes as L_1 .
 - Reduction: Removing the nodes in L_1 and all their outgoing edges from G transforms G into the reduced graph G' .
 - Reintegration: By the inductive assumption, G' has a valid layered topological sort $L' = \{L_2, L_3, \dots, L_k\}$. Therefore, prepending L_1 gives $L = \{L_1, L_2, \dots, L_k\}$, which must be a valid layered topological sort of G .
 - Next subset: With L_1 removed, a new set of nodes becomes the sources in the reduced graph G' , forming the next layer L_2 in the ordered collection.
 - Further Reduction: Removing the nodes in L_2 and their outgoing edges further reduces G' . Repeating this process until no nodes remain results in an ordered collection of subsets of nodes $\{L_1, L_2, \dots, L_k\}$ where each set L_i is the set of source nodes identified in each iteration and k is the number of reductions required until G' has no nodes.

From this construction, for any node $v \in L_i$, all parents $p \in \text{Pa}(v)$ must lie in earlier layers L_j with $j < i$. That is, when processing layer L_i , all parent dependencies are already resolved, so each node is processed exactly once without revisiting. Therefore an algorithm on a DAG that processes nodes in this order is guaranteed to converge in exactly $|L|$ iterations. $\square$

This guarantee matters specifically because every analysis in this framework is an exact analytical method without iterative convergence loops. Given this, declaring a fixed processing order once allows for reuse across the framework. Furthermore, because nodes within the same processing layer share no dependencies, the layers provide a natural parallelisation boundary for any algorithm that supports it.

4.3.2 Extended Network Properties

The graph builder extends the parsed minimal object into IPF's complete graph object. Beyond the iteration sets just established, downstream methods also rely on cheap ancestry and reachability lookups, which the neighbourhood dictionaries alone do not provide. In large networks, computing these additional properties can be expensive, so the builder constructs them once during the same topological traversal.

In this work, the iteration sets L of G are considered one such property and are defined within the unified graph object. The input module computes this property of the network by implementing a depth-partitioned variant of Kahn’s algorithm as shown in Algorithm 1.

Algorithm 1 FindIterationSetsWithClosure

Require: Directed acyclic graph (acyclicity validated upstream by parser) as edge list E , outgoing index Out , incoming index In

Ensure: iteration_sets , ancestors , descendants

```

1: if  $E = \emptyset$  then
2:   return  $\langle \rangle, \emptyset, \emptyset$ 
3: end if
4: compute  $\text{in\_degree}[v]$  for every node  $v$ 
5: initialise  $\text{ancestors}[x] \leftarrow \{x\}$  and  $\text{descendants}[x] \leftarrow \emptyset$  for every node  $x$ 
6: enqueue all source nodes (those with  $\text{in\_degree} = 0$ ) into queue  $Q$ 
7:  $\text{iteration\_sets} \leftarrow \langle \rangle$ 
8: while  $Q$  not empty do
9:    $\ell \leftarrow |Q|$   $\triangleright$  snapshot queue size = one topological level
10:   $\text{current\_set} \leftarrow \emptyset$ 
11:  for  $i = 1$  to  $\ell$  do
12:     $x \leftarrow \text{dequeue}(Q)$ 
13:    add  $x$  to  $\text{current\_set}$ 
14:    for all  $y \in \text{Out}[x]$  do
15:       $\text{ancestors}[y] \leftarrow \text{ancestors}[y] \cup \text{ancestors}[x]$ 
16:      propagate  $y$  as a descendant of every node in  $\text{ancestors}[x]$ 
17:       $\text{in\_degree}[y] \leftarrow \text{in\_degree}[y] - 1$ 
18:      if  $\text{in\_degree}[y] = 0$  then
19:        enqueue  $y$  into  $Q$ 
20:      end if
21:    end for
22:  end for
23:  append  $\text{current\_set}$  to  $\text{iteration\_sets}$ 
24: end while
25: return  $\text{iteration\_sets}, \text{ancestors}, \text{descendants}$ 
    
```

While layering itself is $\mathcal{O}(|V| + |E|)$, tracking the full ancestor and descendant closure within the same pass introduces additional set-propagation cost that depends on the graph’s transitive density. This is a deliberate space-for-time trade-off, justified by the type of set operations the framework performs repeatedly. In the worst case, each node’s closure could contain every other node, giving $\mathcal{O}(n^2)$ total space.

In return, any reachability query reduces to a single set membership test in $\mathcal{O}(1)$, rather than incurring an on-demand $\mathcal{O}(|V| + |E|)$ traversal per query.

Since both sets are populated within the same Kahn’s pass, space is the only additional cost incurred beyond the topological sort itself.

4.3.3 The Unified Graph Object

The framework's unified graph object is a named tuple assembled at the end of input processing, combining the validated outputs of the parser and the graph builder into a single canonical representation. This object is the handoff point from input processing to analysis. For any downstream algorithm (probability propagation, capacity analysis, critical path, diamond decomposition) to be invoked, all arguments can be derived from this tuple. The components of the unified object are split between the shared graph specific fields and the optional analysis-input fields. This is to allow for the same instance of a network to be evaluated under different scenarios even for the same analysis type.

As shown in Table 4.2, the data structure choices throughout reflect the operations the framework actually performs at runtime.

Adjacency indices like the `outgoing_index` and `incoming_index` are stored as `Dict{Int64, Set{Int64}}` because unlike Vectors, Julia's dictionary type gives $O(1)$ average-case lookup without requiring contiguous integer node identifiers. On the other hand, the Set-typed values used in fields like `fork_nodes` and `join_nodes` were chosen to efficiently handle order-insensitive properties that need membership, union, and cardinality operations performed on them.

Tuples are stack-allocated and immutable in Julia so the `Tuple{Int64, Int64}` edge list carries no per-edge heap allocation. Additionally, tuples in Julia are hashed by value so the same `(source, target)` pair maps directly to its entry in any edge-keyed dictionary without auxiliary indexing needed. Iteration sets are stored as `Vector{Set{Int64}}`, since the outer vector position ensures the sets are ordered, but internally each set holds its level members in a container with no positional ordering.

4.3.4 Why not Julia's Native Graph Object?

The Julia language natively supports graph modelling and operations through the `Graphs.jl` package. Component-level information can be attached through complementary packages such as `MetaGraphs.jl` and `NamedGraphs.jl`, all of which are widely used and well-maintained.

At instantiation, IPF immediately validates input data directly from a file path. The input processing module's parser was designed to automatically handle file variation beyond extension type such as excluding header rows and comment lines.

By contrast, building a digraph in `Graphs.jl` with the same input would need to strip these manually because it can only be constructed from a Julia typed matrix or an edge iterator. Since `Graphs.jl` does not have a DAG specific constructor,

Table 4.2: Components of the Unified Graph Object

Component	Type	Role
<i>Base graph — built once, shared across all analyses</i>		
edgelist	Vector{Tuple{Int64,Int64}}	Ordered edge sequence; input to Kahn’s in-degree counting
outgoing_index	Dict{Int64, Set{Int64}}	Forward adjacency: node $\rightarrow$ children
incoming_index	Dict{Int64, Set{Int64}}	Reverse adjacency: node $\rightarrow$ parents
source_nodes	Set{Int64}	Nodes with no incoming edges; DAG roots
sink_nodes	Set{Int64}	Nodes with no outgoing edges; DAG leaves
all_nodes	Vector{Int64}	Sorted list of all nodes in the graph
fork_nodes	Set{Int64}	Nodes with fan-out > 1
join_nodes	Set{Int64}	Nodes with fan-in > 1
iteration_sets	Vector{Set{Int64}}	Kahn BFS levels; dependency-safe processing order
ancestors	Dict{Int64, Set{Int64}}	Transitive upward closure (includes self)
descendants	Dict{Int64, Set{Int64}}	Transitive downward closure
<i>Analysis inputs — loaded per invocation, optional</i>		
node_priors	Dict{Int64, T}	Prior value per node; $T \in \{\text{Float64, Interval, pbox}\}$
edge_probabilities	Dict{Tuple{Int64,Int64}, T}	Conditional probability per edge
edge_capacities	Dict{Tuple{Int64,Int64}, Float64}	Flow capacity per edge
node_capacities	Dict{Int64, Float64}	Flow capacity per node
node_values	Dict{Int64, T}	CPM node-level weight; $T \in \{\text{Float64, Interval}\}$
edge_values	Dict{Tuple{Int64,Int64}, T}	CPM edge-level weight

the acyclicity check is no longer guaranteed by the parser and will have to be performed separately.

Although both approaches require an integer-keyed node set, without companion packages such as `NamedGraphs.jl`, `Graphs.jl` enforces a contiguous integer vertex set where our module does not.

Moreover, the iteration sets in the custom object are grouped by topological depth, unlike `Graphs.jl`'s flat ordering. While the flat order is technically a valid topological sort, it does not expose the level structure required for independent operations within a layer which would force all propagation sequences within the framework to always be sequential even when they can be parallelised.

IPF's custom object defines node classifications such as source, sink, fork, and join nodes directly. While `Graphs.jl` exposes degree information, these roles would have to be inferred every time they are needed. Similarly, since `Graphs.jl` does not expose a precomputed transitive closure, any reachability query would require a fresh traversal each time.

These derived properties can technically be added on top of a base `SimpleGraph` by extending it with `MetaGraphs.jl`, attaching iteration sets, closure dictionaries, and node classifications as graph-level properties. This is workable but stretches the package beyond its intended use, since `MetaGraphs.jl` was designed for node-level and edge-level attributes rather than whole-graph structural products. Either way, the framework still needs to compute these properties itself before storing them anywhere.

Additionally, `MetaGraphs.jl` stores properties in Symbol-keyed dictionaries with loosely typed values, which is intentional for flexibility but introduces type instability and triggers dynamic dispatch on every access. Because the input module is purpose-built for this framework, it can enforce strict typing throughout, which gives Julia's compiler more concrete information than a generic metadata dictionary. In the lookup-heavy set operations performed repeatedly during propagation, the custom structure's concrete field types let the compiler resolve these operations statically.

The motivation behind the custom object is the runtime contract: every downstream algorithm receives a validated DAG with iteration sets, closure, and node roles already computed and attached, accessed through one consistent interface. Building this contract requires the same custom logic regardless of whether the base layer is a `SimpleGraph` or a purpose-built object. Thus, using the packages would only redistribute that work into adapter code wrapping the package objects.

The framework's single canonical object also avoids the consistency risk of maintaining derived structures separately from the topology they describe.

Finally, as a general-purpose graph library, Graphs.jl ships with algorithms and utilities the framework does not use, which adds load-time and maintenance cost without operational benefit. More broadly, fewer external dependencies mean less installation friction and a smaller maintenance surface for users adopting the framework.

Chapter 5

Sub-graph Decomposition: Diamond Processing Module

5.1 Introduction

The Input Module returns a validated graph object that carries the network topology together with its layered iteration sets and ancestry closures. All analysis modules in the framework consume this object directly as is.

The Network Decomposition Module, when invoked, extends the graph object with the network’s decomposition structure. Like input processing, this is a network pre-processing step whose objective is to identify all instances of a specified subgraph pattern in the input DAG network.

5.1.1 Decomposition in Complex Networks

Real infrastructure networks are rarely trees. Their components connect to and depend on many others at once, and the interactions between them overlap rather than branch cleanly [1, 2]. Redundancy strategies are the obvious case: a robust network deliberately supplies more than one route to the same goal, so paths diverge and reconverge by design. Process networks and hierarchical flows behave the same way, with a component feeding several downstream dependencies while itself depending on shared upstream outputs. As the system grows, overlapping paths accumulate and the connectivity web thickens.

Graph-based analysis methods almost always do some form of decomposition, and for large networks with complicated connectivity it is a practical necessity [3]. Broadly, decomposition breaks a network into substructures that are easier to handle than the whole, and methods differ in what they do with those substructures. Some strategies aim to reduce: a substructure is replaced by a single equivalent element, so the network shrinks while the analysis stays exact. Other decomposition methods work by partition: the network is divided into units of work sized for the solver, and the pieces are processed in turn. Beyond either, decomposition can give a more compact representation of the network, storing a repeated substructure once rather than every time it appears, without losing the overall structure. Depending on how the decomposed units are defined, they can also support localised investigation, such as confining a sensitivity study to one part of the system without needing to run analysis against the full network.

There is no universal correct unit of decomposition; a decomposition method defines its own target substructure and partitions the network in those terms. In the classic series–parallel reduction, the reliability formula replaces each series or parallel pair with a single equivalent component and the network shrinks pair by pair [4]. The units here are the series and parallel pairs themselves. When the network is not series–parallel, that reducible structure is missing, and the factoring approach manufactures it: conditioning on a single component splits the

problem into the component-working and component-failed cases, and the method recurses on each until it becomes reducible [5–7]. Polygon-to-chain reductions extend the same idea, widening what can be collapsed in one step beyond the plain series and parallel pairs [8].

In dynamic programming the unit is the tree: a tree decomposition arranges the network into small overlapping bags of nodes connected in a tree, each interacting with the rest of the graph only through its neighbours, which is exactly the subproblem structure a dynamic program can process in a single sweep [9]. Working over bags rather than the whole graph is what makes exact reliability computation feasible on networks as large as metropolitan transit systems [10].

The framework’s default decomposition unit is the *diamond*: a fork–join subgraph where paths in the network fan out from a common node and fan back in at a node further downstream. IPA’s decomposition is partition-forward, that is, each identified diamond is returned as a localised graph object of the same form as the full network, and can be handed to any analysis module in the framework on its own. Secondly, a diamond can serve as a reductive unit, since a solved diamond can stand in for its subgraph as a single equivalent component.

5.2 Preliminaries and Definitions

Identification works directly on the unified graph object of Chapter 4. From it the module reads the DAG $G = (V, E)$ and the iteration sets $L = \{L_1, \dots, L_k\}$, whose validity is the Convergence Guarantee proved there. It also reads the ancestry closures, where the ancestor closure $\text{Anc}(v)$ is self-inclusive by construction so that $v \in \text{Anc}(v)$ for every node, and the fork and join classifications, a fork having fan-out above one and a join fan-in above one. The node value map travels with the object, but identification reads it in one place only, the exclusion rule of Section 5.4.7. The edge value map plays no part at all and is accepted only so the module’s signature matches the rest of the framework.

Definition 5.1 (Topological index). *Given the iteration sets L , define $\text{topi} : V \rightarrow \{1, \dots, |V|\}$ by numbering nodes in increasing order across layers and in sorted node order within each layer. Every node receives a distinct integer, so topi is a strict total order on V refining the layered order.*

Definition 5.2 (Conditioning context). *A conditioning context $E \subseteq V$ is a set of nodes treated as fixed while assessing the relatedness of their descendants. Identification starts from $E = \emptyset$ and extends the context one node at a time as it recurses.*

Definition 5.3 (Unconditioned influence set). *For a node p and context E ,*

$$\text{infl}(p, E) = \text{Anc}(p) \setminus E.$$

Two parents p, q of a join are structurally correlated under E when $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$, that is, when some node upstream of both is not held fixed by the context. When the intersection is empty no unfixed node can reach both parents, and the two incoming paths are structurally unrelated.

Definition 5.4 (Diamond). *A diamond is a triple $D = (R, C, \mathcal{E}_D)$. The relevant node set $R \subseteq V$ spans the reconvergent pattern. The conditioning set $C \subseteq V$ records which nodes identification fixed, or found already fixed, to isolate the pattern. The edge list $\mathcal{E}_D \subseteq E$ is induced on R by the membership rule constructed in Section 5.4.5 and characterised in Lemma 5.9.*

The module returns diamonds at two levels. A `DiamondsAtNode` record attaches to one join node its diamond together with the join’s *non-diamond* parents, meaning the parents found unrelated to every reconvergent group at that join. The map `root_diamonds` collects these records for every join of G at the top level, and the diamonds recorded there are the network’s *maximal diamonds*, identified at the network’s own joins rather than inside another diamond. Separately, every diamond identified at any level, top-level or nested, is stored exactly once in `unique_diamonds`, keyed by the structural identity of Section 5.4.6. Each entry carries the diamond’s fully analysed local structure (Section 5.6), and nested reconvergence appears as entries referencing other entries, so the store is a hierarchy rather than a flat list.

5.3 A Configurable Target Subgraph

The algorithms of this module are written against a definition of the special subgraph, not against the diamond in particular. Three predicates carry that definition. `is_det` decides when a node’s reachability counts as already settled, `shared_fork` decides what counts as a correlating structure worth conditioning on, and `components` with `infl` decides when two parents count as independent. The identification and build algorithms do the same work whatever these predicates say. They find every subgraph of the input network whose structure satisfies the supplied properties, so a user can redefine the target around the network characteristics they care about and the module as it stands will decompose the network in those terms.

Apart from the `is_det` test, identification never reads the value map, and the structures it emits only carry the map restricted to their spans. The decomposi-

tion therefore behaves identically under all three of the framework’s value types (deterministic, interval, and p-box), which is required of every module sitting between input processing and the analysis toolkits.

The diamond is the framework’s default target, and as shipped the predicates encode the definitions of Section 5.2.

5.4 The Identification Algorithm

The identification procedure `new_identify` is the first of the module’s two algorithms. It visits every join node of G once at the top level and recursively inside diamonds, and Algorithm 2 shows its recursive skeleton.

Algorithm 2 Recursive diamond identification (skeleton)

Require: join node v , context E , admissible parent set P

Ensure: list of `DiamondsAtNode` for v under E

- 1: partition P into groups by pairwise $\text{infl}(\cdot, E)$ intersection ▷ union-find;
Section 5.4.2
 - 2: **for all** groups G with $|G| \geq 2$ **do**
 - 3: $f \leftarrow$ covering candidate of G with minimum topi ▷ Section 5.4.3
 - 4: construct diamond D for (v, G, f) under boundary $B = E \cup \{f\}$ ▷
Section 5.4.5
 - 5: look up D in `unique_diamonds` by its identity key; reuse if present ▷
Section 5.4.6
 - 6: **for all** join nodes j interior to D **do**
 - 7: recurse on $(j, B, \text{parents of } j \text{ within } D)$ ▷ residual reconvergence
 - 8: **end for**
 - 9: **end for**
 - 10: singleton groups become non-diamond parents of v
-

5.4.1 Conditioning only where correlation exists

By Definition 5.3, two parents of a join are correlated under E exactly when their unconditioned influence sets intersect. Extending the context with a node removes that node from every influence set, which is the factoring move of Section 5.1.1 applied inside a single network. The algorithm conditions only where an intersection actually exists and only on one node at a time. Nothing is enumerated exhaustively, and recursion stops as soon as the parents in view are pairwise unrelated.

5.4.2 Independence partitioning

At join v with admissible parents P and context E , the procedure `components`(P, E) evaluates $\text{infl}(p, E) \cap \text{infl}(q, E)$ for every pair and merges classes by union-find whenever the intersection is non-empty. The groups it returns are the connected components of the correlation relation (Theorem 5.6), and distinct groups share no unconditioned ancestry at all (Theorem 5.7), so each group can be decomposed on its own. Parents in singleton groups are related to nothing at v and become the join's non-diamond parents.

5.4.3 Fork selection

For a correlated group G , `shared_fork`(G, E) tallies every node that *covers* at least two members of the group. Coverage arises in two ways. A fork node may lie in the ancestry of two or more members, which is the symmetric picture of a diamond. A member of G may instead be an ancestor of another member, the asymmetric case where the reconverging paths share no third node and the covering node is one of the parents itself. Both are genuine reconvergence and both are tested. Candidates already in E are skipped, since fixing an already fixed node does nothing, and so are candidates excluded by `is_det` (Section 5.4.7).

Among the surviving candidates the algorithm selects the one with minimum topological index, the covering node closest to the sources. Conditioning as far upstream as possible leaves the largest part of the group's shared structure intact downstream of the chosen node, where it stays available for reuse. Because `topi` is a strict total order the minimum is unique.

5.4.4 The recursive loop

Selecting fork f for group G at join v produces the diamond, constructed under the extended boundary $B = E \cup \{f\}$. It does not end the work at v , because conditioning on f need not resolve all correlation inside the pattern. Any interior join whose parents remain correlated under B gives rise to a nested diamond, found by the same procedure with the extended context. The recursion bottoms out when every group everywhere is a singleton, and Theorem 5.5 bounds its depth by $|V|$.

5.4.5 Edge construction and isolation

The diamond for group G at join v under boundary B spans

$$R = \{v\} \cup G \cup \bigcup_{p \in G} \text{Anc}(p),$$

the join, the group, and everything upstream of the group, with induced edge list

$$\mathcal{E}_D = \{ (u, w) \in E : u, w \in R, w \notin B, (w \neq v \text{ or } u \in G) \}.$$

The first two clauses confine the diamond to its span. The third cuts every edge into a boundary node, since a conditioned node's upstream structure is fixed and must not be re-entered, while its outgoing edges remain and feed the pattern as a local source. The fourth clause admits an edge into the join only from the group itself, and it exists to close a failure mode that the first three clauses cannot see.

Consider a nested level at which an enclosing diamond has already conditioned on a fork r , so $r \in E$, and suppose r is both an ancestor of some group member p and a direct parent of the join v . The partitioning of Section 5.4.2 handles r correctly. With $r \in E$ its influence set is empty, so it lands in a singleton group and its contribution to v enters separately, fixed by the enclosing context. The span R is built differently, from the raw closure $\text{Anc}(p)$, which contains r no matter what the context says. The partitioning works with context-filtered ancestry while the span works with raw ancestry, and the two silently diverge at every nested level, exactly where $E \neq \emptyset$. The edge (r, v) then has both endpoints in R and a head outside B , so the first three clauses admit it, and a diamond meant to capture v as reached through G alone quietly gains a second unrelated entry into its own join. The fourth clause blocks exactly this edge. Figure 5.1 shows the minimal instance, and the trace of Section 5.7.2 confirms that the implementation behaves as the clause requires, with the excluded parent surviving as a non-diamond parent and its direct edge to the join appearing in no diamond edge list.

5.4.6 Context-aware identity

Every diamond is identified in `unique_diamonds` by the key

$$\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap R)),$$

its edge list together with its *relevant context*, meaning the fork it conditions on plus whatever already conditioned nodes lie in its span. The second term is what

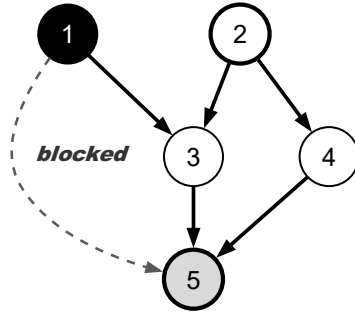


Figure 5.1: Edge isolation at join 5 in context $E = \{1\}$. Node 1 (filled) was conditioned by an enclosing diamond and is excluded from the group $G = \{3, 4\}$ with fork 2. Its direct edge $(1, 5)$, drawn dashed, is admitted by the first three filter clauses but blocked by the fourth. Bold edges form the nested diamond’s edge list.

makes deduplication safe. The structural case that forces it involves a fork that is also a graph source. Such a node has no incoming edges, so the boundary cut of Section 5.4.5 does nothing to it, and the same edge list arises whether that fork is being conditioned at the current level or was already fixed one level up. The two situations are different objects, one a top-level diamond and the other nested inside the diamond that did the fixing, and a key of the form $(\mathcal{E}_D, \{f\})$ cannot tell them apart. Proposition 5.11 proves both that the naive key genuinely collides and that the relevant-context key separates exactly the cases that must be separated.

5.4.7 The exclusion predicate `is_det`

One rule in identification consults declared node values. A node is excluded from conditioning candidacy when its reachability is already settled, which covers exactly two cases. A dead node, declared value 0, is never active, so nothing is reached through it. A certain source, declared value 1 with no incoming edges, is always reachable and has nothing upstream to resolve. The conjunction in the second case matters. A value-1 node that is not a source is still uncertain as a destination, because its reachability depends on its incoming edges, and it must remain conditionable. The simpler test of excluding every node valued 0 or 1 would disable every diamond in a network whose node values all happen to be 1, a configuration that real benchmark scenarios do use. For interval-valued and p-box-valued networks the predicate is conservatively false. It is one of the three override points of Section 5.3, and a user can replace it with an exclusion rule of their own.

5.5 Structural Correctness

All results in this section are statements about the structure of the input graph and nothing else. They divide along the line drawn in Section 5.3, with termination and determinism belonging to the recursive skeleton and grouping, disjointness, and completeness to the predicates that define the diamond.

Theorem 5.5 (Termination). *The recursion of Algorithm 2 terminates after at most $|V|$ nested conditioning steps along any recursive path.*

Proof. Along any path of the recursion, each call either returns without recursing, which happens when every group at the join in view is a singleton, or selects a fork f and recurses with $B = E \cup \{f\}$. The candidate collection of Section 5.4.3 never offers a node already in E , so $f \notin E$ and $|V \setminus B| = |V \setminus E| - 1$ strictly. The quantity $|V \setminus E|$ is a non-negative integer that strictly decreases along the recursive path, and from its initial value $|V \setminus \emptyset| = |V|$ it admits at most $|V|$ decrements. $\square$

Theorem 5.6 (Grouping correctness). *Fix a join v , context E , and parent set P . Let H be the undirected graph on vertex set P with an edge between p and q if and only if $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$. Then $\text{components}(P, E)$ returns exactly the connected components of H .*

Proof. The procedure evaluates the intersection test for every unordered pair in P , which by construction is precisely the edge relation of H , and performs a union–find merge exactly on the pairs that pass. Union–find computes the partition of a vertex set into connected components given its edge relation. Applied to the vertices and edges of H , its output is the component partition of H . $\square$

Theorem 5.7 (Group disjointness). *If $G_i \neq G_j$ are distinct groups returned by $\text{components}(P, E)$, then*

$$\left(\bigcup_{p \in G_i} \text{infl}(p, E) \right) \cap \left(\bigcup_{q \in G_j} \text{infl}(q, E) \right) = \emptyset.$$

Proof. Let $p \in G_i$ and $q \in G_j$ be arbitrary. By Theorem 5.6 the two lie in different connected components of H , so (p, q) is not an edge of H . Since every pair was tested directly, the test itself must have failed, giving $\text{infl}(p, E) \cap \text{infl}(q, E) = \emptyset$. The displayed intersection expands to the union of these pairwise intersections over all $p \in G_i$ and $q \in G_j$, and every one of them is empty. $\square$

Disjointness is a statement about unconditioned influence sets, and it is what permits decomposing the groups separately. Whether separately computed values

recombine correctly at the join is a different claim, which Chapter 6 makes and proves in its own terms.

Theorem 5.8 (Global completeness). *Let v be a join node of G with parent set P , and suppose there exist distinct $p, q \in P$ and a node a with $\neg \text{is_det}(a)$ such that either a is a fork node in $\text{Anc}(p) \cap \text{Anc}(q)$, or $p \in \text{Anc}(q)$ (the asymmetric case, taking $a = p$). Then the top-level pass of `new_identify` stores a diamond for v in `root_diamonds`. Conversely, a join admitting no such pair receives no entry.*

Proof. At the top level $E = \emptyset$, so in either case $a \in \text{infl}(p, \emptyset) \cap \text{infl}(q, \emptyset)$ and the intersection is non-empty. By Theorem 5.6, p and q lie in a common group G with $|G| \geq 2$. In the candidate tally of Section 5.4.3, a is counted once for each member it covers, so from p and q alone its coverage is at least 2 and the candidate set is non-empty. A non-empty finite candidate set has a topi-minimal element, so `shared_fork` returns some fork, not necessarily a itself, Algorithm 2 constructs a diamond for G , and the top-level loop records it under v . For the converse, if no such pair exists then every pairwise influence intersection at v is empty, every group is a singleton, and the algorithm constructs nothing at v . That is the correct outcome, because the incoming paths of v are pairwise structurally unrelated. $\square$

Lemma 5.9 (Edge isolation). *Let D be constructed for group G at join v under boundary B as in Section 5.4.5. Then $(u, w) \in \mathcal{E}_D$ if and only if $(u, w) \in E$, $u, w \in R$, $w \notin B$, and $(w \neq v \text{ or } u \in G)$. Consequently every directed path within $\mathcal{E}_D$ that terminates at v has its final edge originating in G , and no path within $\mathcal{E}_D$ passes through a node of B .*

Proof. The membership characterisation restates the filter defining $\mathcal{E}_D$. For the path properties, induct on path length. A length-one path into v is a single edge $(u, v) \in \mathcal{E}_D$, which the fourth clause forces to have $u \in G$. Any longer path has a final edge satisfying the same property, and each preceding edge (u, w) has $w \notin B$ and both endpoints in R by the first three clauses, so no intermediate node of the path lies in B and no edge of the path enters from outside R . $\square$

Theorem 5.10 (Determinism). *For a fixed input graph and iteration sets (G, L) , join v , context E , and admissible parent set, the diamonds returned by Algorithm 2 are uniquely determined, independent of set or dictionary iteration order, thread scheduling, or any other accident of execution.*

Proof. By induction on recursion depth, it suffices that each step of one call is a function of its arguments alone. This holds for the partition step because,

by Theorem 5.6, its output is the connected-component partition of H , a graph invariant that does not depend on the order in which pairs are examined or merges performed. It holds for the selection step because within each group the algorithm returns the minimum of a non-empty finite subset of V under topi , and a strict total order (Definition 5.1) admits exactly one minimum on such a set. Non-emptiness is guaranteed whenever the group has size at least two, by the tally argument of Theorem 5.8. The boundary $B = E \cup \{f\}$ and span R are then determined, hence so is $\mathcal{E}_D$, and hence so are the interior joins passed to the recursive calls, which are deterministic by the inductive hypothesis. $\square$

The shape of this guarantee, a fixed total order pinning every choice point so that the output is a canonical function of the input, is also the shape of the canonicity guarantee of reduced ordered binary decision diagrams [11]. The mechanism is not the same. BDD canonicity depends on an externally supplied variable ordering, whereas topi is derived from a property of the input graph already proved in Chapter 4.

Proposition 5.11 (Context-aware identity: necessity and sufficiency). *(i) There exist graphs, a source fork f , and contexts $E_1 = \emptyset$ and $E_2 \ni f$ under which the construction of Section 5.4.5 yields the same edge list $\mathcal{E}_D$, while the resulting diamonds belong at different levels of the nesting hierarchy. An identity of the form $(\mathcal{E}_D, \{f\})$ therefore conflates structurally distinct diamonds. (ii) Under $\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap R))$, two constructions receive the same key only if they have the same edge list, condition on the same fork, and their contexts agree on every node of the span R . In that case they pose the identical sub-problem and deduplication is correct.*

Proof. (i) Let f be a fork that is also a graph source, and let v be a join reachable from f along two paths. If f is free and selected at the current level, so that $E_1 = \emptyset$, the boundary cut removes the edges into f , of which there are none, and produces some edge list $\mathcal{E}_D$. The diamond is top-level. If instead f was conditioned by an enclosing diamond, so that $f \in E_2$, the same cut is applied for the same reason, again removes nothing, and produces the same $\mathcal{E}_D$. This diamond is nested inside the one that fixed f . The pair $(\mathcal{E}_D, \{f\})$ is identical in the two situations, but the diamonds are not interchangeable, since one must be recorded at the top level and the other inside its enclosing diamond's substructure.

(ii) Suppose $\text{hkey}(D_1) = \text{hkey}(D_2)$. Equality of edge lists forces equality of the spanned node sets R , and equality of $\{f\} \cup (E \cap R)$ then forces the two contexts to agree on every node of that shared span. Nodes outside R cannot affect the diamond's interior, because by Lemma 5.9 no path of $\mathcal{E}_D$ enters from outside R , so a node beyond the span has no route by which to influence the pattern whether

it is conditioned or not. Two constructions that agree on the edge list, the fork, and the context restricted to the span therefore define the same sub-problem, and sharing one stored entry between them loses nothing. In the collision of part (i) the enclosing construction had conditioned at least one node lying in the span, namely f itself, so $E_1 \cap R \neq E_2 \cap R$ and the keys differ. The two diamonds that must be kept apart are kept apart. $\square$

The key is also deliberately coarse in one direction. Constructions whose contexts differ only outside the span receive the same identity, and part (ii) is the reason this is reuse rather than a collision.

Taken together, these guarantees have a counting consequence. Consider a join whose k parents are all mutually correlated in the naive sense, with every parent sharing some ancestry with some other. Conditioning on the shared cutset jointly means enumerating its 2^k assignments. The decomposition replaces this in two moves, both already proved. The parent set is partitioned into groups whose influence sets are disjoint (Theorems 5.6 and 5.7), so the groups decompose independently. Each group, whatever its size, is then reduced to a single conditioning fork, one fork per group rather than one per correlated pair. In the extreme case where partitioning finds k separate groups, k constant-size single-fork problems stand where a 2^k enumeration stood.

The identification pass itself is polynomial. Per join it computes pairwise influence intersections under union-find and builds one span, and the `unique_diamonds` store bounds the number of distinct diamonds ever materialised. The exponential object in this problem is the downstream conditioning enumeration, and the decomposition's contribution is to make those enumerations as small and as widely reused as the network's structure allows.

5.6 Representation and Reuse

5.6.1 Diamonds as graph objects

Identification finds the diamonds. The module's second algorithm builds each one into a graph object of the same form as the full network. Each entry of `unique_diamonds` is a `DiamondComputationData` carrying the complete local structural analysis of one diamond, and its fields correspond one-for-one to the unified graph object of Chapter 4, as Table 5.1 shows.

Because a diamond satisfies the same object contract as the network it came from, anything the framework can do with the full network it can do with a diamond, including decomposing it further, which is what the recursion does.

Table 5.1: Field correspondence between the unified graph object of Chapter 4 and `DiamondComputationData`

Unified graph object	Diamond entry	Role
<code>outgoing_index</code>	<code>sub_outgoing_index</code>	Forward adjacency
<code>incoming_index</code>	<code>sub_incoming_index</code>	Reverse adjacency
<code>source_nodes</code>	<code>sub_sources</code>	Local sources of the span
<code>fork_nodes</code>	<code>sub_fork_nodes</code>	Local forks
<code>join_nodes</code>	<code>sub_join_nodes</code>	Local joins
<code>ancestors</code>	<code>sub_ancestors</code>	Closure restricted to the span
<code>descendants</code>	<code>sub_descendants</code>	Closure restricted to the span
<code>iteration_sets</code>	<code>sub_iteration_sets</code>	Layered order restricted to the span
<code>node_priors</code>	<code>sub_node_priors</code>	Value map restricted to the span
—	<code>sub_diamond_structures</code>	Nested diamonds interior to this one

The analysis recorded in Table 5.1 is paid for once at identification time and reused on every later consultation.

The reductive use of a diamond noted in Section 5.1.1 rests on the same contract. A solved diamond can stand in the parent graph as a single equivalent node precisely because it satisfies that contract. Identification establishes only the compatibility, since performing the substitution needs a solved value, which structure alone cannot supply.

5.6.2 Deduplication by hash-consing

Before constructing a new entry, the algorithm looks the diamond’s key up in `unique_diamonds`. A structurally identical diamond already resolved anywhere else in the recursion, under another maximal diamond or at another nesting level, is referenced rather than rebuilt. This is hash-consing in the standard sense [12, 13], a table keyed by structural identity so that equal structures are stored once and identity comparison becomes a key comparison. The same technique appears as the unique table of a BDD package. Without the store the recursion is a tree and every call site is distinct. With it, structurally identical call sites collapse into shared entries and the representation becomes a DAG. The collapse loses nothing, because by Proposition 5.11(ii) two call sites receive the same key only when they pose the identical sub-problem.

5.7 Worked Examples

5.7.1 The atomic diamond

Figure 5.2 shows the minimal pattern, a source s feeding a fork f , two internally disjoint paths α and β , and a join j where they meet. At j both parents' influence sets contain f , so the parents form one group. Fork f is the only covering candidate, it is selected, and the diamond spans the whole figure with $C = \{f\}$.

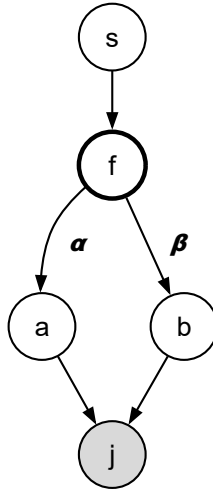


Figure 5.2: The atomic diamond.

5.7.2 A nested decomposition

The eight-node network of Figure 5.3 has forks 1 and 3 and joins 6 and 8. An outer reconvergence runs through fork 1 and meets at join 8, and on one of its branches an inner reconvergence runs through fork 3 and meets at join 6. The decomposition below is not hand-derived. It is the output of the implementation on this edge list, machine-checked, and the example of Figure 5.1 was verified in the same run.

Join 8, top level. The parents are $\{2, 7\}$, with $\text{infl}(2, \emptyset) = \{1, 2\}$ and $\text{infl}(7, \emptyset) = \{1, 3, 4, 5, 6, 7\}$. The sets intersect in $\{1\}$, so the parents form one group. Fork 1 is the only candidate covering both and is selected, and the maximal diamond D_1 spans all nine edges with $C = \{1\}$ (Figure 5.4).

Join 6, inside D_1 . Under the boundary $B = \{1\}$, interior join 6 has parents $\{4, 5\}$ with $\text{infl}(4, \{1\}) = \{3, 4\}$ and $\text{infl}(5, \{1\}) = \{3, 5\}$. The parents are still correlated through fork 3, so conditioning on 1 did not resolve the inner reconvergence, and this is the case the recursion exists for. The recurs-

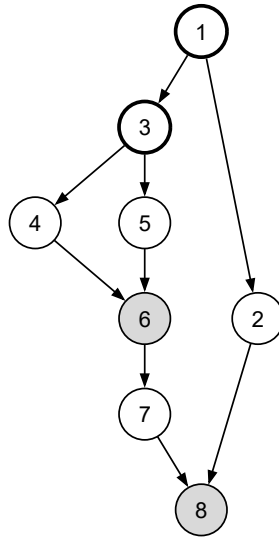


Figure 5.3: Frame 1. The network, with forks outlined in bold and joins shaded.

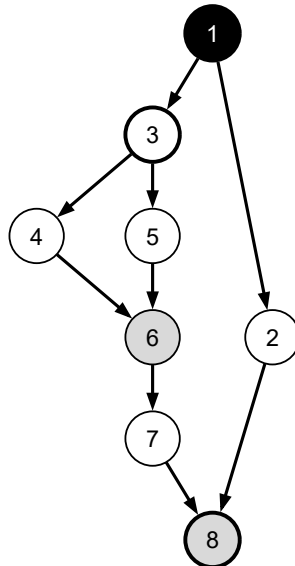


Figure 5.4: Frame 2. The maximal diamond at join 8, with fork 1 conditioned (filled) and the span covering the whole network.

ive call selects fork 3 and produces the inner diamond D_3 with $C = \{3\}$ and $\mathcal{E}_D = \{(3, 4), (3, 5), (4, 6), (5, 6)\}$. The edge $(1, 3)$ is cut by the boundary clause because its head is conditioned at this level (Figure 5.5).

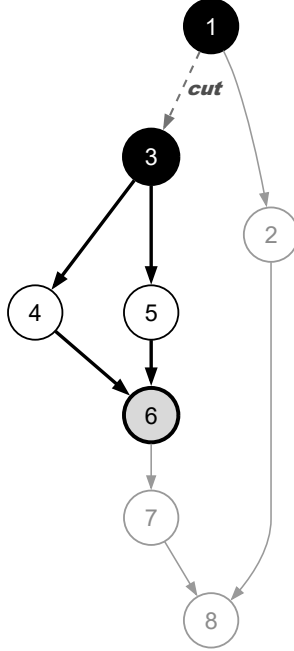


Figure 5.5: Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 selected, the inner diamond in bold, and edge $(1, 3)$ cut.

Join 6, top level. Join 6 is also a join of the network itself, so the top-level pass visits it in its own right, and here the candidate set is larger. With $E = \emptyset$, forks 1 and 3 both cover parents 4 and 5, and the minimum-topi rule of Section 5.4.3 selects fork 1, the covering node closest to the source, rather than fork 3, which a reader tracing by eye might expect. The result is a second maximal diamond D_2 spanning $\{(1, 3), (3, 4), (3, 5), (4, 6), (5, 6)\}$ with $C = \{1\}$, and its interior recursion, now in context $\{1\}$, again produces exactly D_3 .

The emitted hierarchy. In total the module returns two maximal diamonds and three unique diamonds (Figure 5.6). D_1 sits at join 8 and D_2 at join 6, each with $C = \{1\}$. The inner D_3 at join 6 with $C = \{3\}$ is stored once and referenced from the substructure tables of both D_1 and D_2 under the same key. Two different enclosing recursions posed the same sub-problem, with the same edge list, the same fork, and contexts agreeing on the span, and Proposition 5.11(ii) is the reason the single shared entry is correct.

The trace also confirms the edge-isolation case of Section 5.4.5 end to end. In the network of Figure 5.1, the nested diamond at join 5 under $E = \{1\}$ is emitted with $C = \{2\} \cup (\{1\} \cap R) = \{1, 2\}$, which is the context-aware identity visible

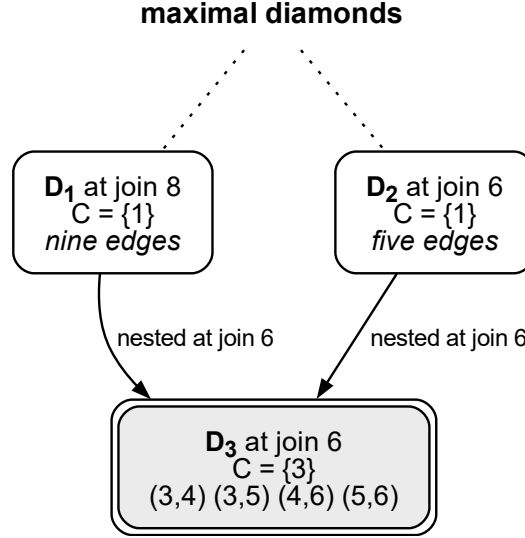


Figure 5.6: Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.

in the output. Node 1 is recorded as a non-diamond parent, and the edge (1, 5) appears in no diamond edge list.

5.8 Summary

The module turns the graph object of Chapter 4 into two structures. `root_diamonds` records the network’s maximal diamonds, and `unique_diamonds` stores every diamond found at any level exactly once, each entry a graph object of the same form as the network it came from. The decomposition terminates within $|V|$ nested steps and is canonical for a given input (Theorems 5.5 and 5.10). It groups a join’s parents exactly by unconditioned shared ancestry and produces a diamond wherever reconvergence exists and only there (Theorems 5.6–5.8). Nothing outside a diamond’s span enters its edge list (Lemma 5.9), and stored structure is shared exactly when sharing is correct (Proposition 5.11). At a join of k correlated parents the result is k single-fork conditioning problems in place of one 2^k enumeration.

Two questions are left open deliberately. One is how the values of independently decomposed groups recombine at a join. The other is the substitution of a solved diamond back into its parent graph as a single equivalent component. Both need values as well as structure, and the Probability Propagation Toolkit takes them up.

Chapter 6

Information As Probabilities: Probability Propagation Toolkit

Chapter 7

Information As Resources: Capacity Flow Toolkit

7.1 Introduction

The reachability toolkit focuses on network survivability and connectivity probability. It computes the probability that at least one active source-to-sink route survives random component failures in a given DAG network.

For some infrastructure reliability problems, this is a sufficient performance measure. In flow-carrying network models, however, the performance measure of the system also depends on the rate of resource flow rather than just the existence of an active path. This reachability module broadly aims to answer the network query of "is the network connected?" However, in networks that also model flow, knowing the likelihood of a signal arriving does not necessarily answer the query of "how much gets through".

Consider a resource distribution network where all source-to-sink paths satisfy the reachability survival threshold. Despite the network being considered connected, deliverable throughput at target nodes can still fall well short of demand if it enters a degraded capacity state. In flow-carrying networks, computing reachability alone cannot fully distinguish between a network with reachability but low capacity and one with reachability plus plenty of spare capacity at target nodes.

Using the fixed edge capacities stored on the extracted graph object in the InputProcessing step, the network flow toolkit consists of a collection of classical max-flow-based modules intended to support analysis of deliverable throughput, bottleneck structure, and the effect of structural or parametric degradation in capacitated DAG models. Specifically, given a capacitated directed acyclic graph as input, the toolkit is designed to address the following analysis objectives:

- Q1.** What maximum throughput can the network deliver under current capacities?
- Q2.** Which cut sets form the binding bottlenecks on that throughput?
- Q3.** Which component failures or capacity perturbations most reduce deliverable flow?
- Q4.** What degradation or upgrade threshold is required to maintain a target demand level?
- Q5.** How much edge and node redundancy exists in path-disjoint terms?
- Q6.** Which components are structural single points of failure?

7.2 Mathematical Model and Network Flow Semantics

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \rightarrow \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modeled, let

$$b : V \rightarrow \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v . Define the flow variable

$$f : E \rightarrow \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u, v))$ for each edge $(u, v) \in E$. For node v , define

$$\delta^+(v) := \{(v, w) \in E\}, \quad \delta^-(v) := \{(u, v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t , subject to edge capacities c_{uv} for $(u, v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. 7.1. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The network performance measure must consider and quantify deliverable throughput from s to t , not just s - t path existence.

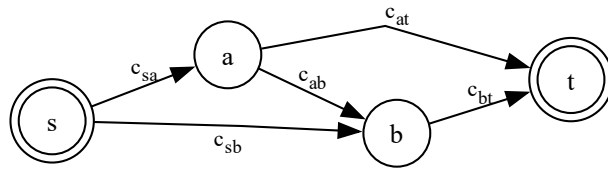


Figure 7.1: Minimal single-source single-sink capacitated DAG network.

For this base case, let $f : E \rightarrow \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t . The optimization model is

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.1)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (7.2)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (7.3)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (7.4)$$

Taking F^* to be the optimal (maximum feasible) value of F . Then if the sink node t has a required demand level D_t , the flow module quantifies service adequacy through the throughput margin

$$\Delta_t := F^* - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

Multi-Source Multi-Sink Reduction

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure 7.2a shows the original multi-source multi-sink network.

By introducing a super-source s^* and a super-sink t^* , as shown in Fig. 7.2b, the reduced graph is

$$V' = V \cup \{s^*, t^*\},$$

$$E' = E \cup \{(s^*, s) : s \in S\} \cup \{(t, t^*) : t \in T\}.$$

The connector capacities are set to non-binding values (implemented as Inf in the solver):

$$c_{s^*s} = +\infty \quad \forall s \in S, \quad c_{tt^*} = +\infty \quad \forall t \in T.$$

Proposition 7.1 (Exactness of the super-terminal reduction). *The optimal value of the multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals s^* and t^* .*



(a) Original multi-source multi-sink network.

(b) Equivalent super-source/super-sink reduction.

Figure 7.2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F .

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^*s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \quad \forall s \in S,$$

and

$$f'_{tt^*} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \quad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, then all capacity constraints from original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from s^* and the net inflow to t^* are both equal to F . Therefore f' is a feasible s^*-t^* flow of value F .

Conversely, let f' be any feasible s^*-t^* flow on G' : then, restricting f' to the original edge set E gives a flow f on G . Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f' . Since feasible flows map in both directions without changing objective value, the two optimal values are equal. $\square$

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.5)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus (S \cup T), \quad (7.6)$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \quad (7.7)$$

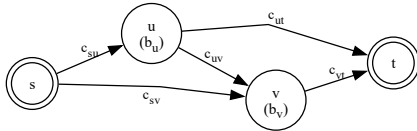
$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \quad (7.8)$$

Node-Capacitated Extension via Node Splitting

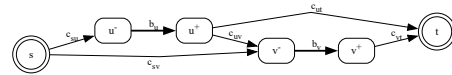
The Information Propagation Framework supports both node and edge network metadata. For capacity analysis, this allows for modelling explicit throughput constraints on nodes themselves. As shown in Figure 7.3, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+ , connected by an internal edge:

$$(v^-, v^+), \quad c_{v^--v^+} = b_v.$$

The original incoming edges of v are redirected to v^- , and every original outgoing edge is emitted from v^+ .



(a) Original network with edge and node capacities.



(b) Equivalent node-split edge-capacitated network.

Figure 7.3: Node-splitting transformation.

This extends the edge-capacitated flow model with

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.9)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c. \quad (7.10)$$

Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimization model then becomes

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.11)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (7.12)$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.13)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.14)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (7.15)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (7.16)$$

7.3 Available Modules

Where required, the flow toolkit applies the super-terminal reduction automatically during the baseline flow solve, and the node-capacitated analysis invokes node splitting internally whenever node-capacity data are supplied. Public outputs are then remapped back to the original graph and original source/sink sets, so edge flows, sink allocations, and cut-side node sets remain interpretable in the user's native network model.

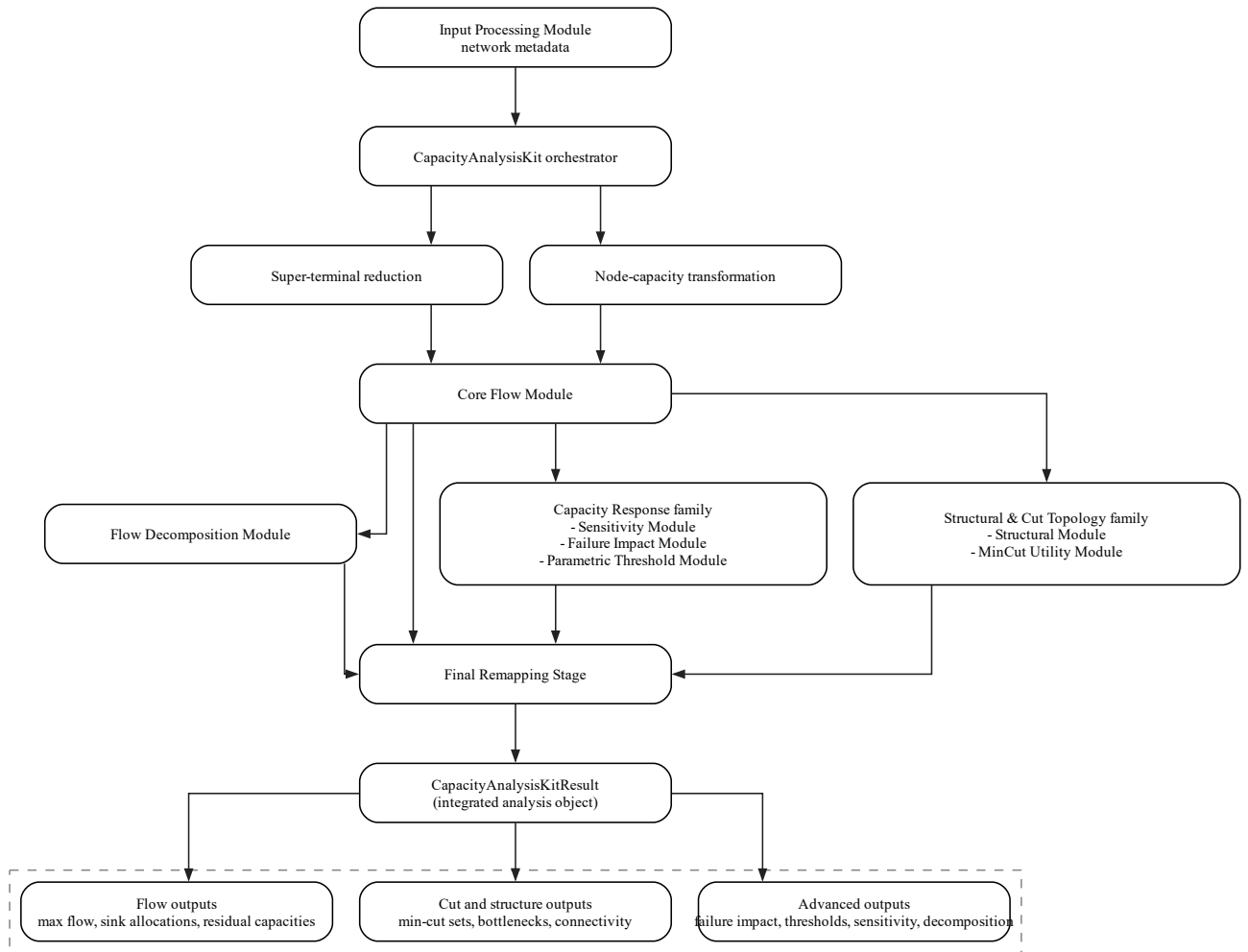


Figure 7.4: Runtime workflow for the flow-capacity toolkit with available modules

7.4 Base Flow Module

Motivating question: “How much flow can the network sustain, and what limits it?”

Inspired by Julia’s `Graphs.jl` and Python’s `NetworkX` flow analysis packages, the flow engine supports three solver variants to compute F^* for a given capacitated network G , summarised in Table 7.1.

Solver	Complexity	Best Fit
<code>:edmonds_karp</code>	$O(VE^2)$	Small networks
<code>:dinic</code>	$O(V^2E)$	Larger layered networks
<code>:push_relabel</code>	$O(V^2E) - O(V^3)^a$	Dense, high fan-out networks

^a Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 7.1: Max-flow solver comparison.

7.4.1 Max-Flow Min-Cut Solvers

The intuition behind Ford-Fulkerson’s [14, 15] Max-Flow Min-Cut (MCMF) theorem is that given a capacitated network, the maximum permissible flow through the network is restricted by the smallest blocking bottleneck set of edges. That is, the maximum possible flow from $s \rightarrow t$ given G is equal to the sum of flow across the min-cut.

Definition 7.2 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$ and a feasible flow f , the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E, f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad (\text{forward residual}), \\ f_{vu} & (v, u) \in E \quad (\text{backward residual}). \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

Algorithm 3 Ford-Fulkerson Min-Cut Max-Flow

```

1: Initialise residual graph  $G_f \leftarrow G$  with zero flow on all edges
2: while there exists an augmenting path  $P$  from source  $s$  to sink  $t$  in  $G_f$  do
3:    $\Delta \leftarrow \min_{e \in P} c_f(e)$  ▷ Bottleneck capacity along  $P$ 
4:   for each edge  $e = (u, v) \in P$  do
5:      $f(u, v) \leftarrow f(u, v) + \Delta$  ▷ Push flow forward
6:      $f(v, u) \leftarrow f(v, u) - \Delta$  ▷ Update reverse edge
7:   end for
8:   Update  $G_f$  to reflect new residual capacities
9: end while
10: Let  $A$  be the set of nodes reachable from  $s$  in  $G_f$ 
11: Min-cut =  $\{(u, v) \mid u \in A, v \notin A\}$ 
12: return Max-flow value  $|f|$ , min-cut set  $A$ 

```

As described in 3, the Ford-Fulkerson-based solvers work by iteratively finding and pushing flow along augmenting paths. Thus, most of the execution time is spent repeating four main lookups:

- Capacity on any edge $u-v$
- Current flow on any edge $u-v$
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Fulkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a

blocking-flow depth-first search. The level graph restricts traversal to edges $u \rightarrow v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor $\text{ptr}[u]$ to track each node's position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework's default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

7.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

Algorithm 4 Push-Relabel Min-Cut Max-Flow

```

1: height[ $s^*$ ]  $\leftarrow |V|$ ;   height[ $v$ ]  $\leftarrow 0 \forall v \neq s^*$ 
2: Push maximum admissible flow from  $s^*$  to all neighbours; initialise excess[ $v$ ]
3: while  $\exists$  active node  $u$  with excess[ $u$ ]  $> 0$  do
4:   if  $\exists$  neighbour  $v$  with  $r(u, v) > 0$  and height[ $u$ ] = height[ $v$ ] + 1 then
5:      $\delta \leftarrow \min(\text{excess}[u], r(u, v))$ 
6:     Push  $\delta$  units from  $u$  to  $v$ ; update excess[ $u$ ], excess[ $v$ ],  $r(u, v)$ ,  $r(v, u)$ 
7:   else
8:     height[ $u$ ]  $\leftarrow 1 + \min_{v: r(u, v) > 0} \text{height}[v]$   $\triangleright$  Relabel
9:   end if
10: end while
11: Traverse final flow state from  $s^*$  to identify min-cut partition
12: return Max-flow value  $|f|$ , min-cut set

```

7.5 Network Flow Diagnostics

7.5.1 Min-Cut Utility Module

Motivating question: “How constrained is the bottleneck, and in how many equivalent ways?”

The flow module solvers all compute the max flow F^* by constructing a representative valid min-cut S^* in the given network G . This output by itself neither indicates nor guarantees that S^* is the only possible min-cut set in G .

This distinction matters in practice for some engineering decisions. Infrastructure systems are designed with robustness and redundancy in mind. In these networks it is more common to expect multiple structurally distinct S/T partitions of V that all have the same max flow F^* value.

While the baseline max flow value directly addresses the first analysis objective, the utility module exploits the already-generated min-cut and residual graph to build the extended partition S^{**} and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from t^* in G_f .

Definition 7.3 (Extended Source Partition). *The extended source-side region is*

$$S^{**} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity. Consequently $S^ \subseteq S^{**}$.*

Definition 7.4 (Free Zone). *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{**} \setminus S^*.$$

Nodes in $\mathcal{F}$ can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of G under f^ takes the form $S = S^* \cup R$ for some $R \subseteq \mathcal{F}$.*

The free zone size $|\mathcal{F}|$ itself determines the degeneracy of the bottleneck structure. Specifically, when $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and S^* is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition S^{**} and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. It then follows that two distinct edge sets can be derived directly from from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^{**}, v \notin S^*\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^*, v \in T^{**}\},$$

where $T^{**} = V \setminus S^{**}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^* \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq \text{cut_limit}$ all minimum cuts are returned and `is_complete = true`; otherwise, the first `cut_limit` cuts are returned with `is_complete = false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

7.5.2 Structure Utility Module

Motivating question: “Where is the network structurally fragile?”

Designed to inform routing and redundancy decisions, the structural module identifies components that are fragile purely due to their position in G , regardless of any capacity assignment.

Understanding the route-level structure of G is a prerequisite for any path-level fragility or flow-distribution analysis. To this end, the module exposes a base path enumerator function that performs a depth-first search on the topologically sorted iteration set derived from the `InputProcessingModule`. Since G is a DAG, exact enumeration is always finite; however, highly redundant networks can admit exponentially many routes, so a `path_limit` parameter bounds the number of enumerated paths and returns a representative sample rather than the complete set $\mathcal{P}$.

Flow Decomposition. Two networks can share identical topology and the same F^* while concentrating flow through entirely different routes. During design decisions where multiple configurations are evaluated and per-route visibility is needed, this decomposition exposes how f^* is distributed across $\mathcal{P}$.

Given an optimal flow f^* and a set of flow-agnostic paths $\mathcal{P}$, for each path $P = (v_0, v_1, \dots, v_k) \in \mathcal{P}$, the bottleneck flow contribution is the minimum flow on any edge along P :

$$\phi(P) = \min_i f_{v_i v_{i+1}}^*,$$

Thus, the edge meeting this minimum must be the path's local bottleneck. Paths are sorted in descending order of $\phi(P)$ as a ranked decomposition of f^* into per-route contributions. Where the min-cut module's ranking informs global capacity upgrade decisions, this ranking informs targeted interventions at the route level. In a network supplying two distinct demand nodes via separate routes, the min-cut module points to the tightest cut edge across the whole network, whereas the decomposition identifies independently which component is constraining delivery to each demand node.

Structural Single Point of Failure Nodes.

Definition 7.5 (Structural Single Point of Failure). *A node $v \in V \setminus (S \cup T)$ is a SPOF node if $F^*(G \setminus \{v\}) = 0$, where removing v also removes all incident edges. A SPOF node is one through which every source-to-sink path must pass; its removal alone disconnects the network regardless of any capacity assignment.*

The candidate set is restricted to nodes lying on at least one s - t path,

$$\mathcal{C} = (\text{fwd}(S) \cap \text{bwd}(T)) \setminus (S \cup T),$$

where $\text{fwd}(S)$ and $\text{bwd}(T)$ are the sets of nodes forward-reachable from S and backward-reachable from T in G respectively. Nodes outside $\mathcal{C}$ lie on no s - t path and cannot be SPOF nodes. For each $v \in \mathcal{C}$, a BFS is run on $G[V \setminus \{v\}]$; if no sink in T is reachable from any source in S , v is recorded as a SPOF node. The function requires one BFS per candidate and runs in $O(|\mathcal{C}| \cdot (|V| + |E|))$ time.

Noted that in a node-capacitated network, a node appearing in the min-cut is not necessarily a SPOF node in this structural sense. A capacity-constrained node enters the cut-space by way of its split edges $(v_{\text{in}}, v_{\text{out}})$ with capacity b_v and is not necessarily a structural choke-point in G . In these networks, a node can simultaneously be a flow-limiting bottleneck and a SPOF node.

7.5.3 Flow Sensitivity Module

Motivating question: *“Which edges most constrain deliverable throughput, and by how much?”*

At optimality, the flow f^* partitions the edge set into two classes: edges carrying strictly less than their capacity, $f_{uv}^* < c_{uv}$, and edges that are fully utilised,

$$f_{uv}^* = c_{uv}.$$

The latter are referred to as *saturated edges* in this work. By the max-flow min-cut theorem, every minimum cut set is made up only of saturated edges; therefore, any augmenting path from S to T must cross at least one saturated edge. Since the binding constraint lies elsewhere, *unsaturated edges* carry slack and are transparent to perturbation — increasing or decreasing their capacity by a small enough amount has little to no effect on F^* . The Flow Sensitivity module focuses only on saturated edges, since they collectively form the capacity-critical boundary of the network. This module exposes the following three functions.

Critical Edge Ranking. An obvious measure of an edge’s failure impact is how much the max flow drops when that edge is disconnected from the network, referred to as edge criticality in this module. The function re-runs the flow solver on a modified network for each candidate. For each saturated edge (u, v) , the critical impact is:

$$\Delta_{uv}^* = F^*(G, c) - F^*(G, c_0^{(uv)}),$$

where $c_0^{(uv)}$ denotes the capacity function with c_{uv} set to zero and all other capacities unchanged. The function returns the edges in descending order of Δ_{uv}^* .

Marginal Capacity Values. Similar to the critical edge ranker, the function is scoped to saturated edges. For each saturated edge (u, v) , it quantifies the incremental gain in F^* per unit of additional capacity,

$$\mu_{uv} = \frac{F^*(G, c_\delta^{(uv)}) - F^*(G, c)}{\delta},$$

where $c_\delta^{(uv)}$ increases c_{uv} by δ with all other capacities unchanged.

In highly redundant networks, bypass routes often exist that can absorb rerouted flow and upgrade priorities are not always clear. In these systems, an edge can rank high for failure impact but have a lower marginal value. The critical edge function identifies which edges are most damaging to lose while the marginal capacity function complements this by identifying which ones are most worth upgrading.

Marginal Range. Both Δ^* and μ are snapshot evaluations relative to the current capacity assignment c . Depending on the network structure, an edge with a low μ under the current inputs may become highly influential under different capacity values. This Marginal Range function evaluates the total range of influence a single edge (u, v) can exert on F^* :

$$\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)}),$$

where $c_{\infty}^{(uv)}$ sets $c_{uv} = \infty$ and $c_0^{(uv)}$ sets $c_{uv} = 0$, and all other capacities fixed. This function is analogous to the classical Birnbaum importance measure $I_B(i) = \partial h(\mathbf{p}) / \partial p_i$ (see [16]). However, the analogy is conceptual since $\mathcal{B}_{uv}$ is a deterministic throughput range, not a probability-weighted sensitivity.

7.5.4 Failure Impact Module

Motivating question: *“How is deliverable throughput affected when components fail or degrade?”*

The failure impact module models outright component failures and degraded operating states.

Single and Combined Edge Failures. For a single failed edge (u, v) , the throughput impact is:

$$\Delta_{uv} = F^*(G, c) - F^*\left(G, c_0^{(uv)}\right),$$

where $c_0^{(uv)}$ sets $c_{uv} = 0$ with all other capacities unchanged and candidate edges are restricted to E_{some} .

Real failure events are not always isolated to a single component. A storm or substation fault in a power network can take down multiple edges simultaneously. Since the interaction between affected edges may open or close re-routing options, the combined impact of k simultaneous failures is not in general the sum of their individual Δ_{uv} impacts.

The k -edge failure function generalises the single failure case to any set of k edges removed simultaneously. For a combination $\mathcal{K} \subseteq E_{\text{some}}$ with $|\mathcal{K}| = k$, the combined impact is:

$$\Delta_{\mathcal{K}} = F^*(G, c) - F^*\left(G, c_0^{(\mathcal{K})}\right),$$

where $c_0^{(\mathcal{K})}$ sets $c_{uv} = 0$ for all $(u, v) \in \mathcal{K}$. All $\binom{|E_{\text{some}}|}{k}$ combinations require one re-solve each. In practice, because $\binom{|E_{\text{some}}|}{k}$ grows rapidly with k , this function is most tractable for $k = 2$ or $k = 3$ depending on the size and density of the network. The function returns a complete ranking of k -edge failure scenarios by throughput impact.

Capacity Degradation Scenarios. Infrastructure components don’t always fail completely. Partial blockage or degraded components are more likely to present as a reduction in c_{uv} rather than complete loss. Additionally, a domain

expert working across multiple network states may want to compare the effect of varying input capacities and operating conditions. This function is intended to be used to efficiently compute the full F^* picture across all scenarios without needing to re-run the input processing or baseline solve steps for each. The degradation function accepts either an explicit override dictionary specifying new capacities for individual edges, or a uniform scale factor $\alpha \in [0, 1]$ applied to all finite capacities,

$$c'_{uv} = \alpha \cdot c_{uv}, \quad \forall (u, v) \in E.$$

7.5.5 Parametric Threshold Module

Motivating question: *“How much capacity change is needed to maintain or breach a target service level?”*

The previous modules treat edge capacities as fixed and analyse the network’s response to perturbations around them. The threshold module inverts this perspective. Given a required demand level $D_t \leq F^*$ to be met, it determines what capacity each edge must have for the network to meet that target delivery level. This makes it a planning and design tool rather than a diagnostic one.

This module exploits a specific structural property of F^* : it is piecewise linear in any single edge capacity c_e with all others fixed. Logically, each linear segment maps to a fixed min-cut partition S^* , over which the threshold satisfying $F^*(c_e) = D_t$ can be recovered analytically. Therefore, breakpoints must occur exactly where the min-cut partition shifts, i.e. where a change in c_e causes a different set of edges to become the binding bottleneck.

Degradation Threshold. For a target edge (u, v) and a required flow level D_t , the degradation threshold $c_{uv}^\dagger$ is the minimum capacity the edge must retain for F^* to remain at least D_t :

$$c_{uv}^\dagger = \inf \{c \in [0, c_{uv}] : F^*(G, c_c^{(uv)}) \geq D_t\},$$

where $c_c^{(uv)}$ denotes the capacity function with c_{uv} replaced by c . The degradation margin $c_{uv} - c_{uv}^\dagger$ is the slack available before the service level is breached. The module ranks edges by ascending degradation margin, so the most vulnerable edges appear first. A small margin can be interpreted as the edge is close to becoming a binding constraint on D_t , while a large margin means the edge can absorb substantial capacity loss without service impact.

The threshold is located by a partition-change recursion. Starting from the known endpoints $c_e = 0$ and $c_e = c_{uv}$, the algorithm evaluates F^* at the midpoint

and compares the resulting min-cut partition to those at the endpoints. If both endpoints share the same partition, the segment is linear and the threshold is solved analytically. If the partitions differ, the interval contains a breakpoint and the recursion descends into the sub-interval containing D_t . This continues until the threshold is isolated to a partition-constant segment, where it is computed exactly. The result is exactly capacity value at which F^* crosses D_t .

Two degenerate cases are handled explicitly. If $F^*(c_e = 0) \geq D_t$, the edge can degrade to zero without breaching the target. If $F^*(c_e = c_{uv}) < D_t$, the target is unachievable even at the current capacity, meaning the bottleneck lies entirely elsewhere and this edge cannot help.

Upgrade Threshold. If $F^* < D_t$ under the current capacity assignment, the upgrade threshold is the minimum capacity (u, v) must have for F^* to first reach D_t . Since the required capacity may exceed c_{uv} , the upper search bound is not known in advance. The algorithm uses a doubling search to bracket the threshold interval, repeatedly doubling the trial capacity until $F^* \geq D_t$, then applies the same partition-change recursion to locate it exactly.

An edge case arises when the edge is not on any minimum cut: expanding its capacity cannot raise F^* regardless of how large c_{uv} becomes. The doubling search detects this when F^* stops increasing before reaching D_t and returns `upgrade_ineffective = true`, indicating that reaching D_t requires addressing a different bottleneck.

When no explicit target is supplied, the module defaults to $D_t = 0.9 \cdot F^*$.

7.6 Exactness, Validation, and Scope

Exactness as used in this chapter is not the same as global combinatorial exhaustiveness. Where candidate-set pruning is applied, exactness describes the results on evaluated candidates only, not globally across all possible edge subsets or perturbation scenarios.

7.6.1 Post-solve validation

Every public solver call with validation enabled applies three post-solve consistency checks. (i) Capacity feasibility: all edge flows satisfy $0 \leq f(u, v) \leq c(u, v)$. (ii) Node-wise conservation: $\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w)$ at all internal nodes. (iii) Max-flow/min-cut consistency: the solved flow value and the extracted cut capacity agree within numerical tolerance. These confirm primal feasibility and dual optimality consistency before derived analyses are interpreted.

7.6.2 Scope of results

Four main scope boundaries apply throughout this chapter. First, the sensitivity and failure-impact modules use candidate sets drawn from the baseline min-cut structure; results are exact on evaluated candidates, and global exhaustiveness over all edges or all combinatorial subsets requires explicit full-space evaluation. Second, the capacity toolkit does not compute $P(\text{performance} \geq T)$ or any probability-weighted reliability index. Third, all analyses operate on a single fixed-capacity instance; time-varying degradation and adaptive re-routing models are outside the scope of this layer. Finally, for complete throughput loss, minimum cuts are the minimal failure modes; for nonzero service targets, the relevant boundary is characterised by threshold margins $c_{uv} - c_{uv}^\dagger$.

7.6.3 Computational Complexity

The total computational cost of any derived analysis is the product of the per-solve cost of the chosen max-flow algorithm and the number of solver calls required. Repeated-solve tasks scale with solve count, which is the dominant cost driver in practice. Per-solve costs are summarised in Table 7.1.

Module	Function	Operation	Complexity
<i>Base Solver</i>			
Base	Edmonds-Karp	BFS augmenting path	$O(VE^2)$
Base	Dinic	Level graph + blocking	$O(V^2E)$
Base	Push-Relabel	Preflow relabeling	$O(V^3)$
Node-Cap	Split transformation	Graph transform	$O(V + E)$
Node-Cap	Solve (split)	Solver on $2V, 2E$	$O(T_{\text{solver}})$
<i>Min-Cut Utility</i>			
Min-Cut	cut_edges	Extract partition	$O(E)$
Min-Cut	edges_some	BFS + filter	$O(E + T_s)$
Min-Cut	edges_every	Reachability	$O(V + E)$
Min-Cut	enumerate	Lattice ($2^{ \mathcal{F} }$)	$O(2^{ \mathcal{F} } \cdot E)^1$
<i>Structural</i>			
Struct	spof_nodes	BFS per candidate	$O(\mathcal{C} (V + E))$
Struct	enumerate_paths	DFS on DAG	$O(\mathcal{P})^2$
Struct	path_flow	Per-path scan	$O(\mathcal{P} \ell)$
Struct	redundancy	Menger (unit cap)	$O(E_c T_s)$
<i>Flow Sensitivity</i>			
Sens	critical_edges	Zero-cap solves	$O(E_{\text{sat}} T_s)$

Module	Function	Operation	Complexity
Sens	marginal_capacity	$+\delta$ solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_range	Range evals	$O(E_{\text{some}} T_s)$
<i>Failure Impact</i>			
Fail	single_edge	Per-edge solves	$O(E_{\text{some}} T_s)$
Fail	k_edge	Per-combination	$O(\binom{ E_s }{k}T_s)^3$
Fail	degradation	Per-scenario	$O(S T_s)$
<i>Parametric Threshold</i>			
Param	degrade	Partition recursion	$O(\lg(c/\varepsilon)T_s)$
Param	upgrade	Doubling + recurse	$O(\lg^2(c/\varepsilon)T_s)$
Param	all_degrade	Per edge	$O(E_c \lg(c/\varepsilon)T_s)$

Table 7.2: Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.

7.7 Network Flow Output Structure

Field	Module	Description
max_flow	Base Solver	Maximum deliverable throughput F^* from source set to sink set.
flow	Base Solver	Final flow on each original edge.
residual_capacity	Base Solver	Spare capacity on each original edge after the solve.
node_flow	Base Solver	Through-flow at each node: outflow at sources, inflow at sinks, conserved flow at transit nodes.
sink_flow	Base Solver	Delivered flow broken down by individual sink.
mincut_S, mincut_T	Base Solver	Source- and sink-side min-cut node partitions.
mincut_capacity	Base Solver	Capacity of the reported minimum cut.
is_unbounded	Base Solver	Whether unbounded max-flow was detected.

Field	Module	Description
<code>saturated_edges</code>	Base Solver	Edges carrying flow at full capacity $f_{uv}^* = c_{uv}$; forms the candidate set for sensitivity and failure impact modules.
<code>node_cap_max_flow</code>	Node-Cap Base Solver	Maximum deliverable throughput F^* under node capacity constraints.
<code>node_cap_flow_drop</code>	Node-Cap Base Solver	Reduction in F^* relative to the edge-only baseline.
<code>saturated_nodes</code>	Node-Cap Base Solver	Nodes whose through-flow reaches the node capacity bound b_v under the node-capacitated solve.
<code>mincut_S_star_star</code>	Min-Cut Utility	S^{**} : extended source partition; nodes that cannot reach the sink in the residual graph.
<code>free_zone_size</code>	Min-Cut Utility	$ \mathcal{F} $: number of free-zone nodes assignable to either partition without affecting cut capacity. Exactly $2^{ \mathcal{F} }$ equivalent minimum cuts exist.
<code>edges_in_some</code>	Min-Cut Utility	E_{some} : edges appearing in at least one minimum cut.
<code>edges_in_every</code>	Min-Cut Utility	E_{every} : edges appearing in every minimum cut; structural single points of failure.
<code>mincut_enumeration</code>	Min-Cut Utility	Bounded enumeration of all valid minimum cuts in deterministic order, with completion flag <code>is_complete</code> and count.
<code>spof_nodes</code>	Structure Utility	Structural single points of failure nodes: $v \in \mathcal{C}$ where every s - t path must pass through v .
<code>enumerated_paths</code>	Structure Utility	Source-to-sink paths $\mathcal{P}$, bounded by <code>path_limit</code> .
<code>path_contributions</code>	Structure Utility	Per-path bottleneck flow $\phi(P) = \min_i f_{v_i v_{i+1}}^*$ ranked in descending order.
<code>critical_edges</code>	Flow Sensitivity	Saturated edges ranked by max-flow drop Δ_{uv}^* when set to zero capacity, in descending order.

Field	Module	Description
marginal_capacity	Flow Sensitivity	Saturated edges mapped to incremental max-flow gain μ_{uv} per unit capacity increase.
marginal_range	Flow Sensitivity	Total range of influence $\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)})$ for each edge.
fail_min_cut_edges	Failure Impact	Representative min-cut edge set: all edges (u, v) with $u \in S^*$ and $v \in T^*$.
single_edge_fail	Failure Impact	Edges in E_{some} ranked by throughput drop Δ_{uv} when set to zero capacity, in descending order.
k_edge_fail	Failure Impact	All k -edge combinations from E_{some} ranked by combined impact Δ_K , bounded by <code>combination_limit</code> .
degrade_results	Failure Impact	Max-flow and saturated edges for each capacity degradation scenario: either uniform scale factor α or edge-wise overrides.
degrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity $c_{uv}^\dagger$ required to maintain target flow D_t , ranked by ascending degradation margin $c_{uv} - c_{uv}^\dagger$.
upgrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity required for F^* to first reach D_t , with <code>upgrade_ineffective</code> flag where applicable.
param_target_flow	Parametric Threshold	Target service level D_t . Defaults to $0.9 \cdot F^*$ if not specified.
param_baseline_flow	Parametric Threshold	Baseline max flow F^* for reference.

Table 7.3: Unified output schema for network flow capacity analysis across all modules.

7.8 Case Study: Synthetic Benchmark Network

7.8.1 Benchmark Network Design

To demonstrate the full range of the flow toolkit, a synthetic benchmark was generated. The network is a multi-source, multi-sink capacitated DAG with 43 nodes, 107 directed edges. The benchmark simulates a distribution system that propagates resources across geographically dispersed locations. As shown in Figure 7.6, it is structured as an 8-layer hierarchy:

- **Origin nodes** (blue): five source nodes, over-provisioned so the supply zone does not form the binding bottleneck.
- **Hub nodes** (green): eight intermediate relay nodes with high fan-out capacities.
- **Processing nodes** (red, node-capacitated) and **bypass nodes** (orange, node-capacitated): the processing corridor carries the bulk of flow; the bypass is a lightweight alternative with tighter node constraints.
- **Trunk and secondary nodes** (purple, brown): distribute flow through a re-converging structure toward the gateway layer.
- **Dispatch gateways** (light blue, node-capacitated): two gateways assigned equal edge capacity of 17 by design.
- **Distribution combiner** (pink): aggregates all gateway flow before fan-out to the sinks.
- **Sink nodes** (grey): seven demand nodes, each with incoming edge capacity 6.

Based on this layered structure, edge and node capacity assignments are chosen to ensure that binding bottlenecks emerge at specific layers under the different analysis methods. Edge capacities in Layers 1–5 are scaled by a factor of 2 relative to the gateway layer. In this way, the gateway layer governs F^* under the edge-only solution. Similarly, to compute a measurable reduction in F^* , the processing node capacities are set to bind to the node-capacitated solve.

To explore competing design alternatives under controlled conditions, two configurations of the same system are presented to evaluate the effect of feeder reinforcement (Figures 7.6a and 7.6b). Both configurations of the network keep the same backbone topology, but represent different design objectives. Therefore, the comparative objective is not to maximize throughput alone, but to test whether robustness structure changes even when headline flow remains unchanged.

Config A proposes a symmetry-preserving design with balanced feeder-edge capacities and a limited direct bypass to sink 4. The capacity is set below the sink’s 6-unit limit such that the bypass delivers partial rather than full service. Config B’s asymmetric gateway structure is designed to produce non-trivial parametric upgrade thresholds and non-zero marginal-range values. This version has larger feeder capacities from secondary nodes into both gateways.

Table 7.4 summarises the key structural differences between the two. The expectation is that two designs may yield the same max-flow value while looking very different in terms of saturation pattern and min-cut landscape due to their differing feeder-capacity policies.

7.8.2 Comparative Results (Config A vs Config B)

The base flow solver run returns $F^* = 37.0$, with `mincut_capacity` = 37.0 for both networks, consistent with the expected maximum feasible flow for the benchmarked system. However, when optimising for a demand target or comparing two network design approaches, F^* alone is not always enough to inform decisions. As shown in Table 7.4, the two networks still differ in structure: Configuration A has 41 saturated edges versus 35 in Configuration B, and 4 valid minimum cuts versus 1. This is also reflected in free-zone size (2 in A, 0 in B). The same throughput under structurally different constraints motivates the use of the extended analysis modules in the toolkit.

Table 7.4: Configuration comparison summary.

Metric	Config A	Config B
F^*	37.0	37.0
<code>saturated_edges</code>	41	35
<code>free_zone_size</code>	2	0
<code>min_cuts</code>	4	1
<code>SP0F_nodes</code>	0	0
<code>upgrade_ineffective</code> (gateway edges)	true	false

From a design perspective, this reflects a trade-off between structural flexibility and tighter bottleneck localisation. In Configuration A, the free-zone nodes are {134, 135}, producing a wider cut lattice where E_{every} and E_{some} differ. The gateway edges $134 \rightarrow 136$ and $135 \rightarrow 136$ appear in E_{some} but not in E_{every} , so neither is a universal failure mode on its own. Cut 4 has only 3 crossing edges while Cut 1 has 13, both at capacity 37 (Table 7.7). In Configuration B, feeder-capacity reinforcement pins the cut to a unique partition: free-zone size collapses to 0 and the gateway edges move into E_{every} . The choice between configurations depends

on the optimisation target: Configuration A offers more structural redundancy and cut diversity, while Configuration B offers tighter bottleneck localisation and actionable upgrade thresholds.

Bypass edge and structural vulnerability. The direct bypass edge $132 \rightarrow 140$ improves `sink_4` service with capacity 3, below the sink-side limit of 6. The same edge raises redundancy scores for all secondary-to-gateway connections from 2 to 3, improving redundancy across the gateway-feeding layer, not only the directly connected nodes.

Without this edge, node 136 is the only SPOF node: every source-to-sink path passes through it and $F^*(G \setminus \{136\}) = 0$. Adding $132 \rightarrow 140$ gives $F^*(G \setminus \{136\}) = 3 > 0$, removing node 136 from the SPOF set, and both configurations report `spof_nodes` = $\emptyset$. However, only `sink_4` has an alternative path, so all other flow still passes through the combiner. Running the capacity degradation module with node 136 removed confirms this: F^* drops to 3, exposing the network’s continued vulnerability to combiner failure despite the absence of a formal SPOF classification.

Node-capacitated analysis. When node capacities are enabled, both configurations still return the same result: a 7-unit loss down to `node_cap_max_flow` = 30.0. The same six Layer 3 nodes {114, 115, 116, 117, 118, 119} saturate in both cases. This matters because it tells the system designer where to act: once these node limits are active, changing gateway-edge capacities does not change the outcome.

The flow loss is not shared evenly across demand nodes (Figure 7.9). `sink_1` (137) falls from 4.0 to 0.0, so it loses service completely. `sink_2` (138) falls from 6.0 to 3.0, while sinks 139–143 stay at their previous levels. Reporting total flow alone misses the main service issue.

To test a direct fix, a single-node upgrade sweep was run on nodes 114–119 (Table 7.5). Any one of these nodes can restore `sink_1` to its edge-only service level with $\Delta b_v = 7.0$, and this also returns total flow to $F^* = 37.0$ in both configurations. The increment is the same for all six nodes, but the final capacity targets are not: nodes 114/115 go to $b_v = 12.0$, nodes 116/117 go to $b_v = 14.0$, and nodes 118/119 go to $b_v = 10.0$. In practice, that means the choice should be made by delivery constraints such as cost, installation effort, and access, not by throughput effect.

Flow decomposition results. Flow decomposition shows that both configurations achieve the same total throughput ($F^* = 37.0$) with very similar route-level

Table 7.5: Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore `sink_1` service under the node-capacitated solve.

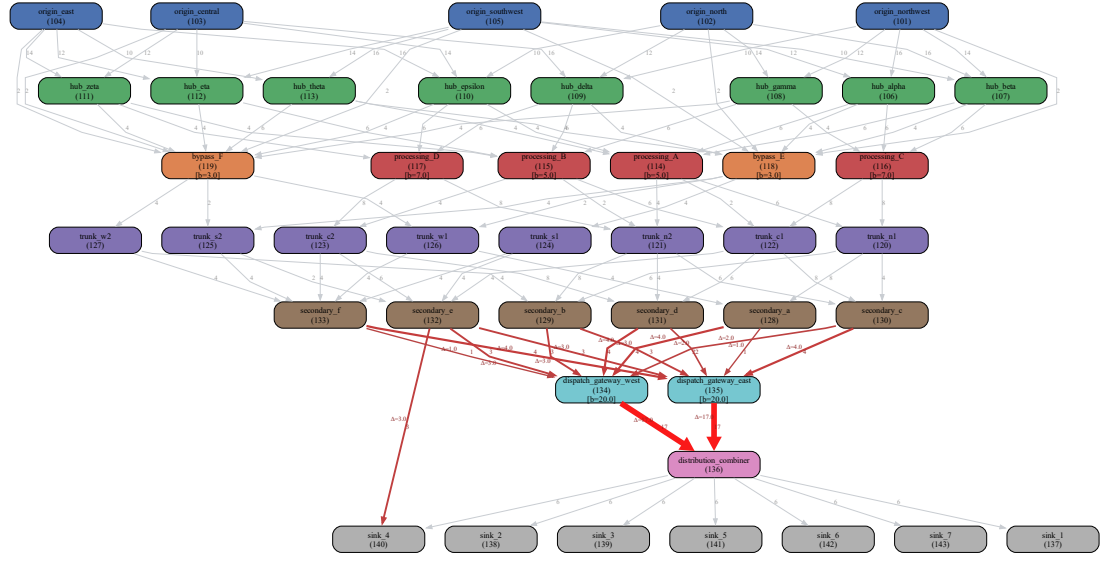
Node	Current b_v	Required Δb_v	New b_v
114 (<code>processing_A</code>)	5.0	7.0	12.0
115 (<code>processing_B</code>)	5.0	7.0	12.0
116 (<code>processing_C</code>)	7.0	7.0	14.0
117 (<code>processing_D</code>)	7.0	7.0	14.0
118 (<code>bypass_E</code>)	3.0	7.0	10.0
119 (<code>bypass_F</code>)	3.0	7.0	10.0

allocation. Configuration A decomposes into 27 components and Configuration B into 25, so the difference is small and does not change the main delivery pattern. In both cases, the largest single component carries 3.0 units on a path bottlenecked at $(114 \rightarrow 120)$, and most remaining components are paths carrying 2.0 or 1.0 units distributed across the processing and bypass corridors.

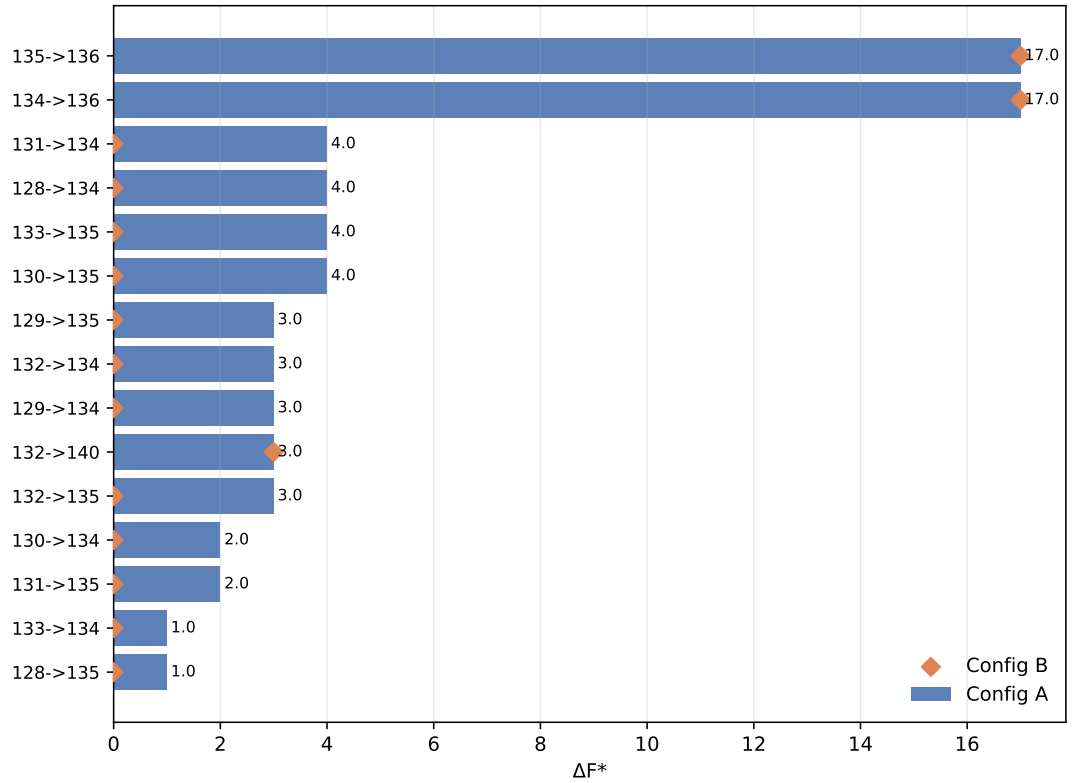
This supports the earlier structural findings. The A/B edge-capacity changes alter which secondary-to-gateway edges appear as local path bottlenecks, but they do not create a different high-level flow-allocation picture. The dominant route family still passes through the same Layer 3 processing/bypass region and then through the combiner to sinks, with the direct bypass to sink 4 contributing only partial service. At decomposition level, the two designs differ in local edge pressure more than in overall route usage.

Sensitivity analysis. Sensitivity ranking shows a clear tiered pattern in Config A (Figure 8 and Table 9). The two gateway-to-combiner edges $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$ rank first and second with $\Delta^* = 17.0$ each, reducing flow from $F^* = 37.0$ to 20.0 when removed (about 46%). The next tier contains four feeder edges with $\Delta^* = 4.0$ and perturbed flow 33.0: $(128 \rightarrow 134)$, $(130 \rightarrow 135)$, $(131 \rightarrow 134)$, and $(133 \rightarrow 135)$. A third tier of five edges has $\Delta^* = 3.0$ and perturbed flow 34.0, including the direct bypass edge $(132 \rightarrow 140)$; this places the bypass in the same sensitivity tier as several secondary-to-gateway feeders. The remaining non-zero edges have $\Delta^* \in \{1.0, 2.0\}$ with perturbed flow 35.0 or 36.0. In total, 15 edges have non-zero single-edge impact in Configuration A.

Configuration B is much more concentrated: only three edges have non-zero impact, $(134 \rightarrow 136)$, $(135 \rightarrow 136)$, and $(132 \rightarrow 140)$, while all other evaluated edges give $\Delta^* = 0$. This means the feeder-capacity reinforcement in B absorbs any single feeder-edge loss through rerouting, so feeder removals no longer reduce F^* individually. The non-zero set therefore collapses from 15 edges in A to 3 in B.



(a) Configuration A critical-edge overlay. Edge emphasis (colour/width) is proportional to non-zero ΔF^* .



(b) Ranked non-zero single-edge sensitivity for Configurations A and B.

Figure 7.5: Single-edge sensitivity in Config A and cross-configuration rank comparison (A vs B).

Failure impact. The two-edge failure ranking confirms that the dominant pair is simultaneous loss of $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$, with drop 34.0 and perturbed flow 3.0. This is much larger than the next group (drop 21.0), and makes sense given the single-edge pattern shown in Figure 7.5. In Configuration B, the same gateway-pair dominance remains, while the non-zero failure set is more concentrated because feeder reinforcement absorbs most single-edge feeder losses through rerouting.

Degradation trajectory. The degradation trajectories are identical for Configurations A and B at all tested scale factors. The solved values are: $\alpha = 1.0 \rightarrow 37.0$, $\alpha = 0.9 \rightarrow 33.3$, $\alpha = 0.8 \rightarrow 29.6$, $\alpha = 0.7 \rightarrow 25.9$, $\alpha = 0.6 \rightarrow 22.2$, and $\alpha = 0.5 \rightarrow 18.5$. These points follow the linear relation

$$F^*(\alpha) = 37\alpha$$

over the tested interval.

The A/B curves that coincide point-for-point (Figure 7.8) indicate that the feeder-capacity differences that separate A and B under single-edge sensitivity do not produce a different global response under a uniformly scaled network-wide degradation.

Parametric thresholds. Parametric thresholds separate edge criticality from edge upgradeability. In Configuration A, the gateway edges $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$ are upgrade-ineffective (`required_capacity` = ∞ , `ineffective`=`true`) even though they are high-impact in sensitivity ranking. In Configuration B, the same gateway edges become upgrade-effective with `required_capacity`=18.0 and `required_increase`=1.0. Degradation thresholds remain at 17.0 with zero margin in both configurations, so any capacity loss below current levels immediately reduces F^* .

This difference is explained by upstream supply. In Configuration A, supply into each gateway is already capped at the current gateway-edge capacity, so increasing gateway-edge capacity alone cannot raise throughput. In Configuration B, feeder reinforcement creates upstream headroom, so gateway-edge upgrades can increase F^* .

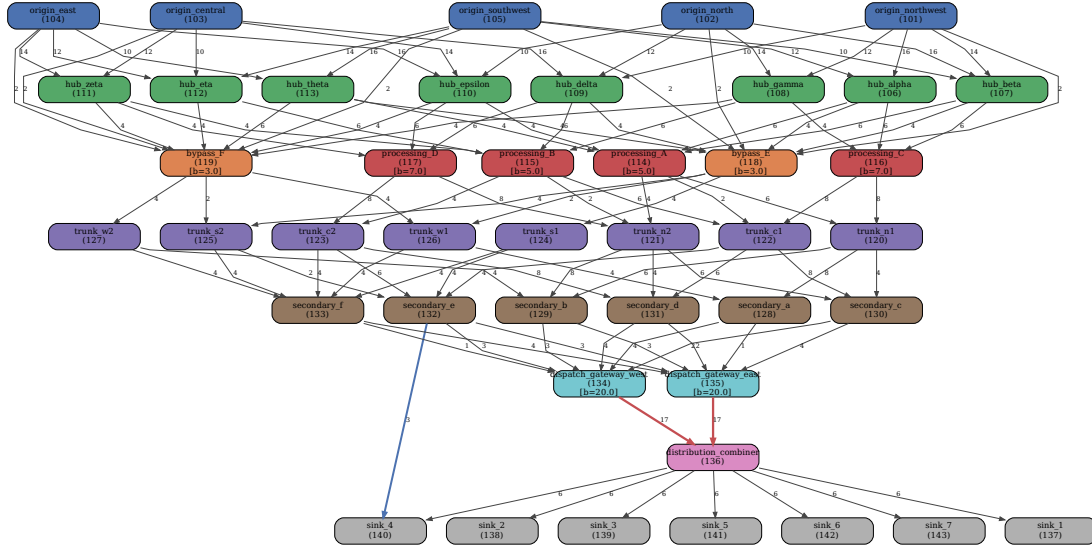
For the bypass edge $(132 \rightarrow 140)$, $\mathcal{B}_{uv}$ increases from 10.0 (A) to 16.0 (B), and the edge is upgrade-effective in both configurations with `required_increase`=1.0 (`required_capacity`=4.0). The main A/B contrast therefore remains at the gateway edges, not at bypass upgrade feasibility.

Table 7.6: Parametric threshold and marginal range comparison for key edges.

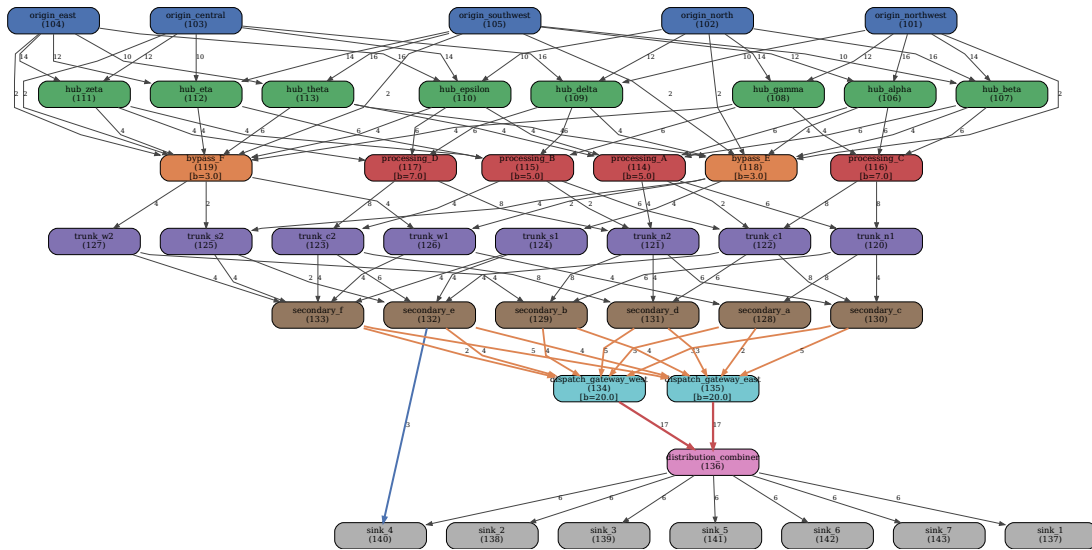
Edge	Config A		Config B	
	Upgrade	$\mathcal{B}_{uv}$	Upgrade	$\mathcal{B}_{uv}$
$134 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$135 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$132 \rightarrow 140$	+1.0	10.0	+1.0	16.0

Table 7.7: Min-cut enumeration for Config A.

Cut	Crossing edges
1	$(128 \rightarrow 134), (128 \rightarrow 135), (129 \rightarrow 134), (129 \rightarrow 135),$ $(130 \rightarrow 134), (130 \rightarrow 135), (131 \rightarrow 134), (131 \rightarrow 135),$ $(132 \rightarrow 134), (132 \rightarrow 135), (132 \rightarrow 140), (133 \rightarrow 134),$ $(133 \rightarrow 135)$
2	$(128 \rightarrow 135), (129 \rightarrow 135), (130 \rightarrow 135), (131 \rightarrow 135),$ $(132 \rightarrow 135), (132 \rightarrow 140), (133 \rightarrow 135), (134 \rightarrow 136)$
3	$(128 \rightarrow 134), (129 \rightarrow 134), (130 \rightarrow 134), (131 \rightarrow 134),$ $(132 \rightarrow 134), (132 \rightarrow 140), (133 \rightarrow 134), (135 \rightarrow 136)$
4	$(132 \rightarrow 140), (134 \rightarrow 136), (135 \rightarrow 136)$



(a) Configuration A: symmetric gateway design. Blue edge $132 \rightarrow 140$ is the direct bypass to `sink_4`.



(b) Configuration B: orange edges show capacity-boosted secondary-to-gateway feeders

Figure 7.6: Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.

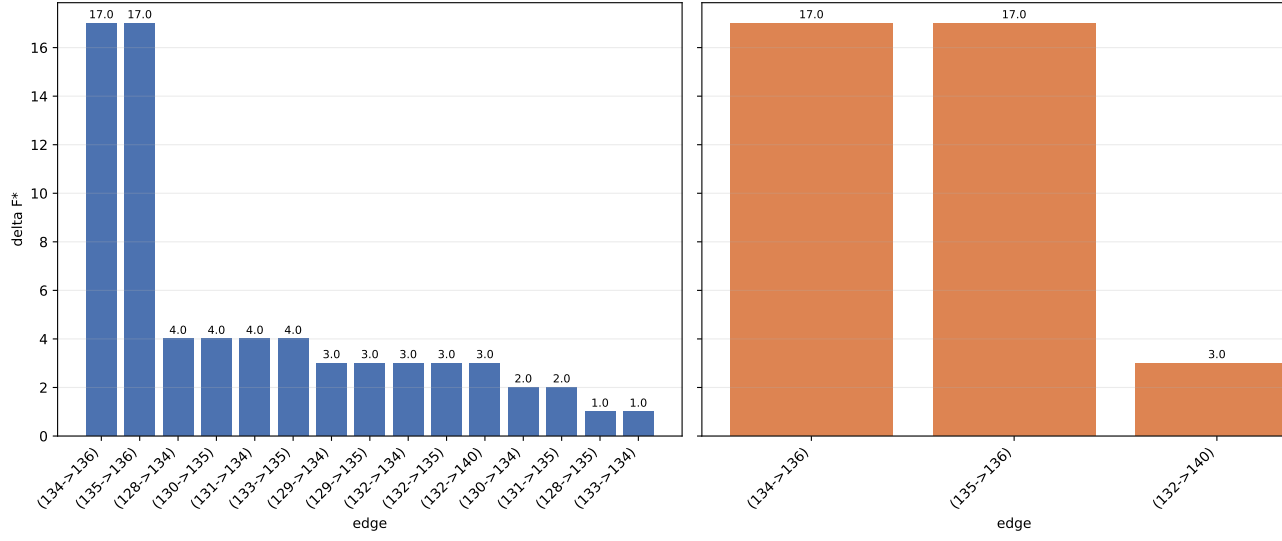


Figure 7.7: Sensitivity ranking for non-zero ΔF^* values in Configurations A and B.

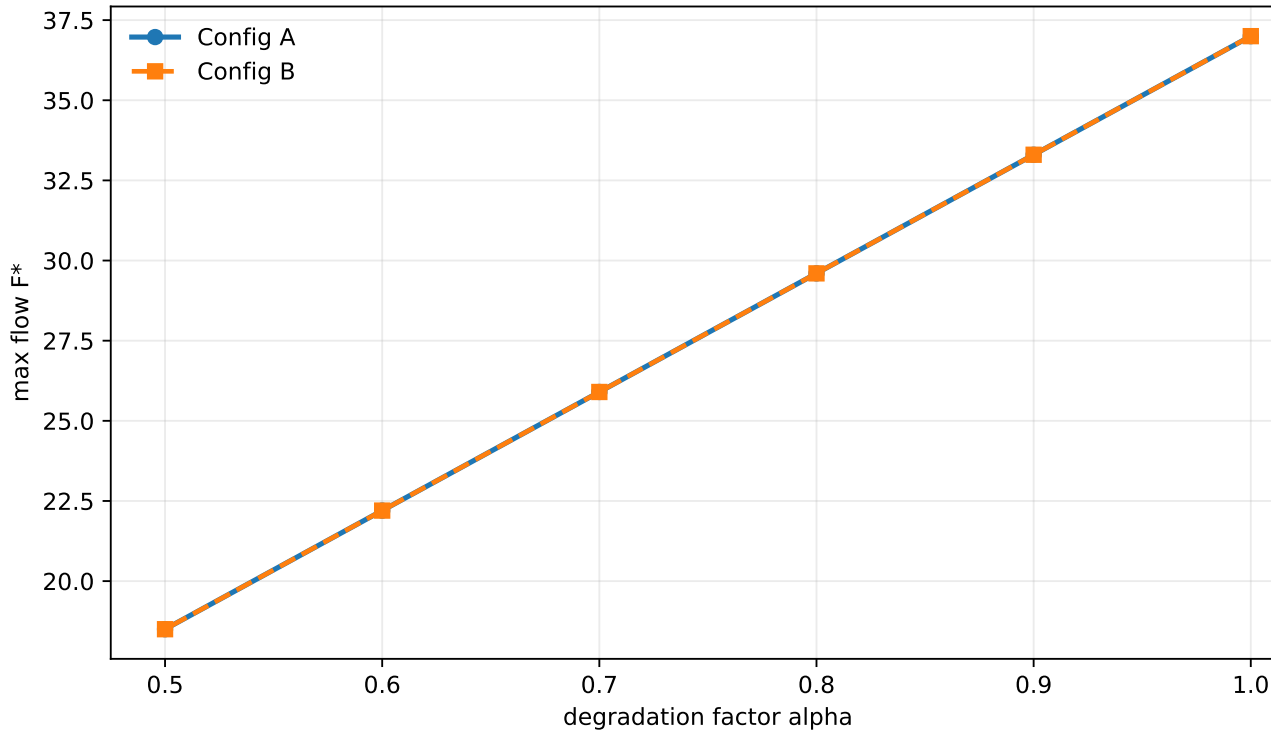


Figure 7.8: Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.

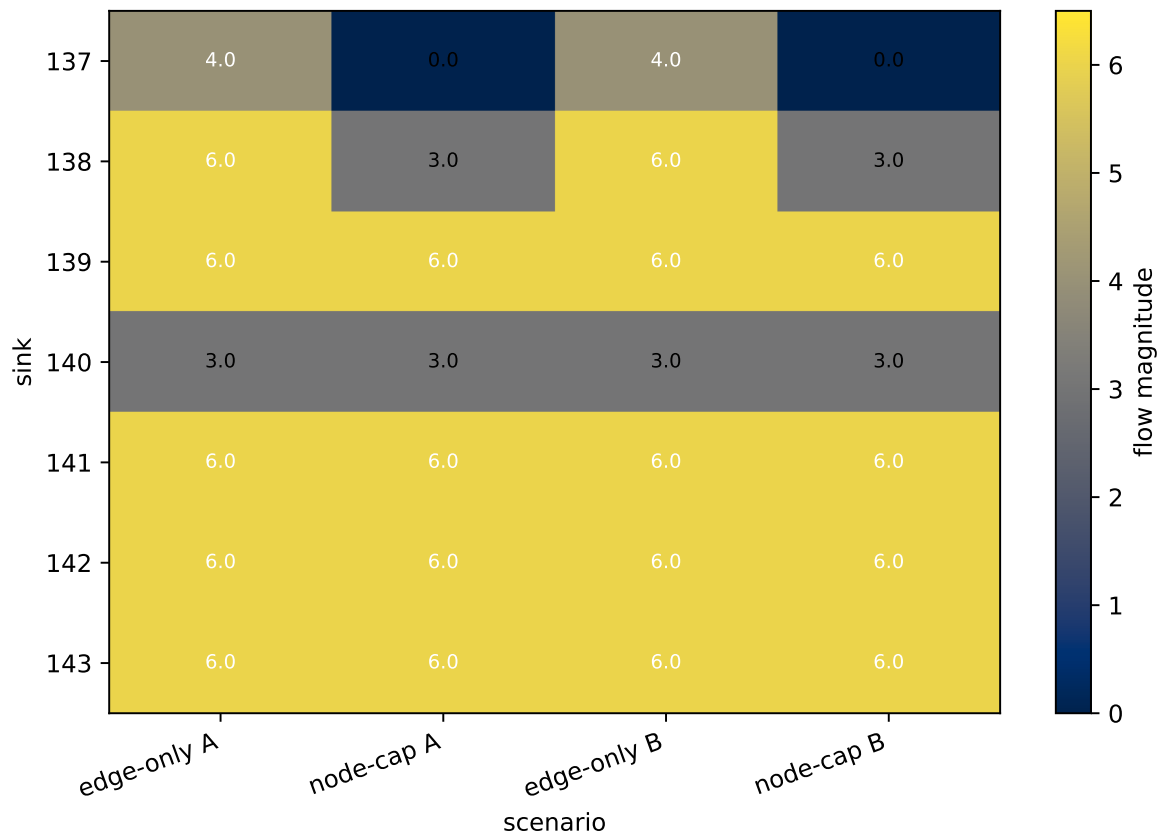


Figure 7.9: Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.

Bibliography

- [1] Ahuja, R.K., Magnanti, T.L., Orlin, J.B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ.
- [2] Birnbaum, Z.W. (1969). On the importance of different components in a multicomponent system. In P.R. Krishnaiah (Ed.), *Multivariate Analysis II*, pp. 581–592. Academic Press, New York. DOI: 10.21236/AD0670563.
- [3] Dinic, E.A. (1970). Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11, 1277–1280.
- [4] Edmonds, J., Karp, R.M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248–264. DOI: 10.1145/321694.321699.
- [5] Ford, L.R., Fulkerson, D.R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399–404. DOI: 10.4153/CJM-1956-045-5.
- [6] Ford, L.R., Fulkerson, D.R. (1962). *Flows in Networks*. Princeton University Press, Princeton, NJ.
- [7] Gallo, G., Grigoriadis, M.D., Tarjan, R.E. (1989). A fast parametric maximum flow algorithm and applications. *SIAM Journal on Computing*, 18(1), 30–55. DOI: 10.1137/0218003.
- [8] Goldberg, A.V., Tarjan, R.E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4), 921–940. DOI: 10.1145/48014.61051.
- [9] Hao, J.X., Orlin, J.B. (1994). A faster algorithm for finding the minimum cut in a directed graph. *Journal of Algorithms*, 17(3), 424–446. DOI: 10.1006/jagm.1994.1043.
- [10] Menger, K. (1927). Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10, 96–115. DOI: 10.4064/fm-10-1-96-115.

- [11] Picard, J.-C., Queyranne, M. (1982). A network flow solution to some nonlinear 0–1 programming problems, with applications to graph theory. *Networks*, 12(2), 141–159. DOI: 10.1002/net.3230120206.
- [12] Whitney, H. (1932). Congruent graphs and the connectivity of graphs. *American Journal of Mathematics*, 54(1), 150–168. DOI: 10.2307/2371086.

Appendices

Table 8: Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).

Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_a->dispatch_gateway_west	4.0	33.0
4	secondary_c->dispatch_gateway_east	4.0	33.0
5	secondary_d->dispatch_gateway_west	4.0	33.0
6	secondary_f->dispatch_gateway_east	4.0	33.0
7	secondary_b->dispatch_gateway_west	3.0	34.0
8	secondary_b->dispatch_gateway_east	3.0	34.0
9	secondary_e->dispatch_gateway_west	3.0	34.0
10	secondary_e->dispatch_gateway_east	3.0	34.0
11	secondary_e->sink_4	3.0	34.0
12	secondary_c->dispatch_gateway_west	2.0	35.0
13	secondary_d->dispatch_gateway_east	2.0	35.0
14	secondary_a->dispatch_gateway_east	1.0	36.0
15	secondary_f->dispatch_gateway_west	1.0	36.0

Table 9: Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).

Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_e->sink_4	3.0	34.0

Chapter 8

Information As Schedule and Cost: Critical Path Toolkit

8.1 Introduction

The Critical Path Toolkit consumes the unified graph object of Chapter 4 directly and involves no decomposition. Every analysis in it is a pair of linear-time passes over the layered iteration sets, so the Network Decomposition Module of Chapter 5 is bypassed entirely.

The quantities this toolkit propagates are the ones that accumulate along dependency paths. A duration, a cost, a risk factor and a load all compound as they travel, and a network carrying such a quantity poses the same pair of questions whatever the quantity is. One is extremal, asking for the best or worst value a complete chain of dependencies can produce. The other concerns room, asking how far an individual component can move before that extremal answer changes. The first is settled by a forward pass. The second is where the analysis actually lives, because room is what criticality, bottlenecks and priorities are made of, and it exists only where the arithmetic of the quantity can be run backwards. This chapter develops that backward reading and its price under uncertainty.

8.2 Scheduling Analysis on Networks

The critical path method models a project as a network of activities and computes the longest path through it. The project cannot finish earlier than that path allows, the activities on it have no room to slip, and every other activity carries a float equal to how far it can slip before it starts to matter. The method and the float calculus date to the late 1950s [17], and the companion question of duration uncertainty was raised almost immediately: PERT replaced each fixed duration with a three-point estimate and a distributional assumption [18], and its descendants in modern practice replace the assumption with Monte Carlo sampling. Distribution-free treatment of the same uncertainty came much later. When each duration is known only as an interval, the forward quantities remain easy, but the backward ones change character entirely. Deciding whether an activity is possibly critical is NP-complete and float bounds are NP-hard [19], even on planar networks [20]. Exact methods exist for latest starting times and one of the two float bounds [21], the consolidated picture is given by Fortin, Zieliński, Dubois and Fargier [22], and the recognised tractable class is the series-parallel networks.

None of the machinery is specific to time. The forward recursion combines incoming values and compounds them along edges, and any quantity with those two operations propagates the same way. With combination as maximum and compounding as addition the recursion is the critical path method. With other

pairs it is cost roll-up, risk scaling or route selection, and the algebra of such systems is well established: max-plus linear systems theory treats the classical forward and backward passes as linear algebra over an idempotent semiring, with the backward pass arising as residuation [23]. The lesson of that algebra is a discipline. A forward recursion exists for any pair of operations, but a backward one is a property the pair either has or lacks, and an analysis that reports slack without checking has manufactured a number. Uncertainty sharpens the same discipline from another side, because the interval results above draw their line between easy and hard exactly where monotonicity is lost, and the structure that loses it turns out to be the reconvergence that Chapter 5 measures for probability.

8.3 Analysis Modes

Whether the room question has an answer is a property of the operator pair, and it has one in exactly two situations. When combination is an order, maximum or minimum, a bound on the combined value binds every argument, and when compounding is monotone with a residual the bound inverts across each edge, so a constraint of the form finish by K travels backwards through the network the same way values travel forwards. That is residuation in an idempotent semiring [23], and it is what the classical backward pass has always been. Summation is different in kind. A total is not a deadline and no residuation exists, but a summed system is linear, and a linear map has an adjoint. The backward object of an accumulation analysis is therefore a sensitivity, and the room it measures is an allowance against a stated budget rather than a slack against a deadline. A pair with neither structure supports a forward recursion and nothing else, and any margin reported for it would be invention. A configuration of the toolkit, called a mode, is an operator pair together with whatever backward semantics its algebra supports, and Table 8.1 lists the four the framework provides.

Table 8.1: The shipped modes. Every row carries a defined backward semantics; the margin column names what the backward pass reports.

Mode	$\oplus / \otimes$	Backward mechanism	Margin
LongestPath	max / +	residuation (min / −)	slack
ShortestPath	min / +	dual residuation (max / −)	margin over optimum
MaxScaling	max / $\times$	residuation (min / $\div$)	ratio slack
Accumulation	sum / +	adjoint (path multiplicities)	allowance

The four margins in the table are two objects, not four. The three order-based modes all report the same thing, the distance between the best complete path

and the best path forced through the node in question, expressed as a difference for the additive pairs and as a ratio for the multiplicative one, and in each case the zero set is the critical structure of the analysis. The allowance is the linear system's counterpart. Because a node's value reaches the target once per route, its influence on the total is a path count rather than a path choice, and the room it has under a budget is the remaining headroom shared across those routes. Running a scheduling backward pass over a summed quantity instead produces figures that answer no question at all, which is why no slack appears in the accumulation row. The water network of Section 8.7.1 shows what the allowance says in its place.

8.4 The Two Kernels

The backward pass is not a second algorithm. For the order-based modes one fold serves both directions. Read forwards over the layered iteration sets it computes, for every node v with parent set $\text{pa}(v)$, duration d_v and edge values w_{uv} ,

$$F_v = \left(\bigoplus_{u \in \text{pa}(v)} F_u \otimes w_{uv} \right) \otimes d_v,$$

the best value a chain of dependencies can deliver into each node, with sources seeded from the initial value. Read over the reversed graph the same fold computes

$$R_v = \bigoplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs},$$

the best completion from each node onwards, with sinks seeded at the neutral element. The two arrays meet in the middle. Their compound $F_v \otimes R_v$ is the best complete path forced through v , the project value P is the best complete path free of any such constraint, and the margin of v is the gap between the two. Slack, in this reading, is the cost of insisting on a node. The textbook schedule quantities are recoveries from the same two arrays, early start as $F_v - d_v$ and late finish as $P - R_v$, with late start and total float following by the same subtractions. On the four-node network of Figure 8.1, with durations 1, 5, 2, 1 and no edge delays, the longest reading completes in 7 along $1 \rightarrow 2 \rightarrow 4$ while node 3 carries slack 3, and the shortest reading completes in 4 along $1 \rightarrow 3 \rightarrow 4$ while node 2 carries margin 3. The same inputs answer opposite questions with opposite critical chains.

Accumulation replaces the order by a sum, and the meeting in the middle changes meaning. The total at a target t expands to $\sum_v m_v d_v + \sum_{(u,v)} m_v w_{uv}$, where m_v counts the directed routes from v to the target, so a node influences the total not through one best path but once per route, and the reverse traversal that computes the counts, seeded with $m_t = 1$, is the adjoint of the forward map. The

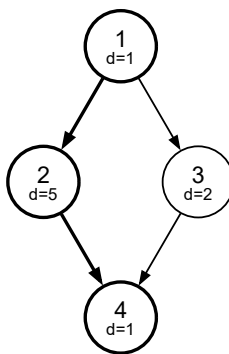


Figure 8.1: The running example. Node durations 1, 5, 2, 1; LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4.

count is the derivative of the total with respect to the node's value, the product $m_v d_v$ is the node's share of the total, and under a budget B the room left to the node is $(B - F_t)/m_v$, headroom divided across its routes. On the running example the total at node 4 is 10 and node 1 counts twice, once through each branch. Both kernels touch each edge a constant number of times, so every mode costs $O(|V| + |E|)$ whatever the operators are.

8.5 Interval Analysis

Uncertainty divides the toolkit's quantities along a single line, and the line is monotonicity. Every forward quantity grows when any input grows, so with interval inputs its exact bounds are two crisp runs of the scalar kernel, one at the lower corner of the input box and one at the upper, and interval arithmetic never enters. Margins sit on the other side of the line. A margin is a difference of two monotone quantities that share their inputs, it is not monotone in anything, and the hardness results of Section 8.2 live precisely there. Running the scalar algorithms on an overloaded interval type would blur this line rather than respect it, ranking overlapping intervals through a partial order that cannot rank them and subtracting dependent quantities as though they were independent. The toolkit instead computes each interval quantity by the scheme its structure permits. Forward quantities are exact for the price of two runs. Margins come either as a sound enclosure derived from those exact bounds, declared conservative in the result it returns, or exactly, and an exact value and an enclosure are never allowed to look alike.

8.6 Exact Interval Floats: the Domination Split

Throughout this section the mode is LongestPath, durations d_u lie in intervals, delays are fixed, and $f_v = P - \text{through}_v \geq 0$ is the float of node v . An interval-valued delay reduces to this setting by subdividing its edge with a node that carries the delay as its duration. The question is the exact range of f_v over the box of duration configurations.

Proposition 8.1 (Corner sufficiency). *With all other durations fixed, f_v is a monotone function of any single duration d_u . Consequently the extremes of f_v over the box are attained at corner configurations.*

Proof. A path visits a node at most once, so as a function of $x = d_u$ every path length is either constant or affine with slope one. Hence $P(x) = \max(C_0, C_1 + x)$, where C_1 is the best path through u and C_0 the best avoiding it, and likewise $\text{through}_v(x) = \max(A_0, A_1 + x)$ over the paths through v . Each has one breakpoint with slope 0 then 1. If P 's breakpoint comes first their difference has slope pattern 0, +1, 0, otherwise 0, -1, 0, and in both cases f_v is monotone in x on the whole line. For the corner claim, push any coordinate to whichever endpoint does not worsen the objective, which monotonicity permits, and repeat per coordinate until the configuration is a corner. $\square$

Lemma 8.2 (Incomparable coordinates). *If no complete path contains both u and v , then f_v is nondecreasing in d_u , whatever the other durations are.*

Proof. No through- v path contains u , so through_v does not depend on d_u , while P is nondecreasing in it. $\square$

Lemma 8.3 (Dominated coordinates). *If every complete path through u also passes v , and in particular if $u = v$, then f_v is nonincreasing in d_u , whatever the other durations are.*

Proof. Suppose increasing d_u increases P . The new maximiser must contain u , hence v , hence it is a through- v path, so through_v rises to equal P and the float drops to zero. If P does not increase, the float cannot rise because through_v is nondecreasing in every duration. $\square$

The two lemmas cover every duration except those in the *bypass set*

$$H_v = \{u : \text{some complete path contains both } u \text{ and } v, \text{ and some complete path contains } u \text{ but not } v\}$$

the nodes that share a route with v yet can also route around it. Figure 8.2 shows the three classes around one node.

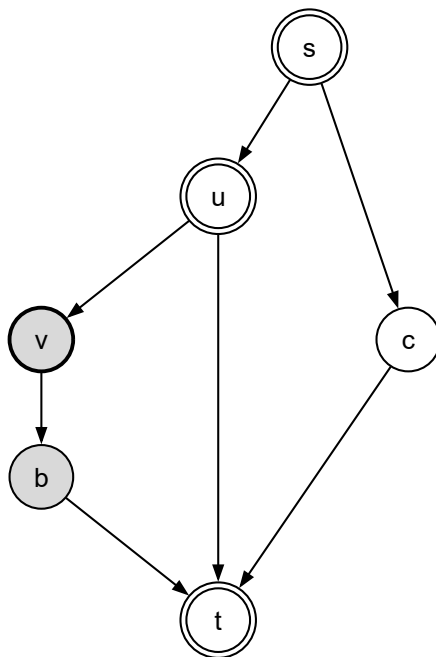


Figure 8.2: Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.

Theorem 8.4 (Exactness of the domination split). *The maximum of f_v over the box is attained at a corner with every incomparable duration at its upper bound, every dominated duration at its lower bound, and the bypass durations at some corner of H_v . The minimum is attained dually. Enumerating the $2^{|H_v|}$ bypass corners with the remaining durations pinned therefore yields the exact float range of v .*

Proof. By Proposition 8.1 an optimum lies at a corner. The three classes partition the durations. At an optimal corner, moving an incomparable duration to its upper bound cannot decrease the maximum by Lemma 8.2, and moving a dominated duration to its lower bound cannot decrease it by Lemma 8.3, so some optimal corner has the pinned pattern and lies within the enumerated set. $\square$

The cost is $\sum_v 2^{|H_v|}$ crisp propagations against 2^k for exhaustive enumeration of all k interval durations, and the exponent has moved from the size of the problem to the reconvergent width around each node. One exhaustive sweep serves every node simultaneously, so the implementation runs whichever of the two is cheaper and is never worse than exhaustive enumeration. Some instances must remain hard, because the general problem is NP-hard [19], and the split shows exactly where the hardness sits: in networks whose bypass sets are large.

The bypass set is a reconvergence measure, and this is where the toolkit touches the structural theme of Chapter 5 without ever calling its module. A bypass node is precisely an N-shaped pattern in the reachability order, and the orders of two-terminal series-parallel networks are exactly the N-free orders [24].

Proposition 8.5 (Series-parallel case). *In a two-terminal series-parallel network every pair of nodes on a common path is in the dominated relation and every other pair is incomparable, so H_v is empty for every v , and the split computes exact interval floats in two crisp runs per node.*

This recovers the known polynomiality of interval criticality on series-parallel networks as a special case, and it makes the kinship with probability precise. The conditioning sets of Chapter 5 and the bypass sets of this chapter vanish on exactly the same class of structures and grow with the same phenomenon. Reconvergence is the price of exactness in both analyses. The two objects are not the same, they index reconvergence per join and per node respectively, and neither computation uses the other. Whether the diamond spans can accelerate the split by sharing corner evaluations between nodes with overlapping bypass sets is an open engineering question.

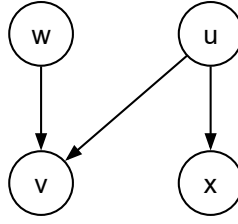


Figure 8.3: The N pattern that places u in a bypass set: u reaches v and also reaches a sink around it. Series-parallel orders contain no such pattern.

8.7 Case Studies

Every number in this section is produced by the toolkit and certified against an independent computation. The certification corpus contains ten networks. On the seven where full path enumeration is affordable, forward values, through-values, margins and critical sets of every mode agree with a brute-force path oracle to within 10^{-15} , and on the remaining three the results pass invariant and sampling checks. Interval results are certified against exhaustive corner enumeration driven by the same path oracle, and the domination split agrees with exhaustive enumeration everywhere both can run.

8.7.1 One network, four questions

The wastewater treatment network of 32 nodes and 66 edges shows how much one set of inputs can say. Its stage processing times and pipeline delays fix the minimum release timeline at 45 units, carried by the chain $3 \rightarrow 11 \rightarrow 19 \rightarrow 27$ with a second chain only 1.5 units behind (Table 8.2). Read for the opposite extreme, the same inputs put the fastest achievable delivery at 15 units, and not along one route. The twelve margin-zero stages and the eighteen tight dependencies between them form a system of tied routes drawing on all three sources, sharing no stage with the critical chain, and the nearest alternative stage sits 1.5 units behind the tie (Figure 8.4). The two extremal readings of one network differ not only in which stages they select but in the shape of what they select, a single thread against a redundant lattice.

Read as load rather than schedule, the same network delivers 480 units to its busiest outlet, and the stages that drive that figure are not the ones the completion times point at. The largest completions sit downstream where load gathers, but the leverage sits upstream at stages 3, 6 and 5, each reaching the outlet along seven distinct routes, so a unit added at any of them arrives seven

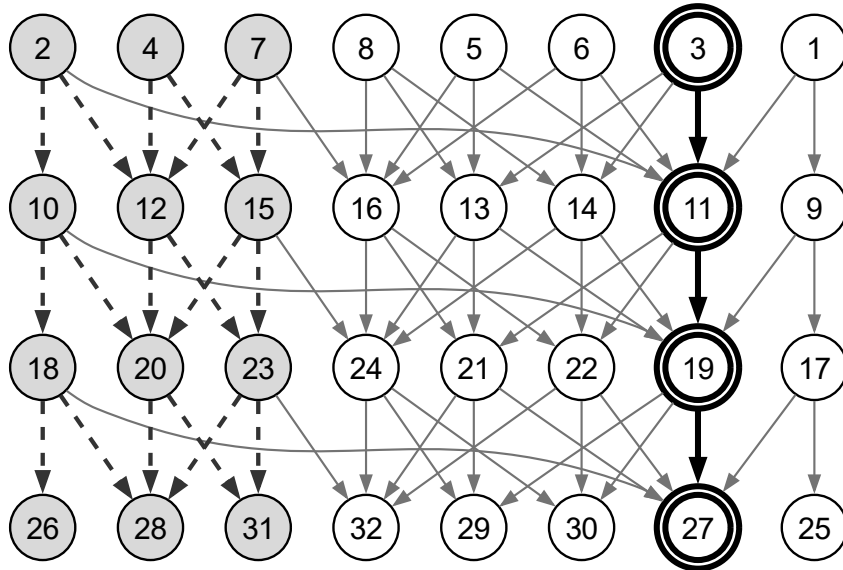


Figure 8.4: The water network read for both extremes. The bold double-bordered chain sets the minimum release timeline of 45 units. The shaded stages and dashed edges form the tied-fastest route system, every path through which completes in 15 units. The two readings share no stage.

Table 8.2: LongestPath on the water network: completion 45 along the critical chain, with the near-critical pair at slack 1.5.

Node	Completion	Slack
27	45.0	0
19	33.0	0
11	21.0	0
3	9.0	0
30	43.5	1.5
14	19.5	1.5

times. Under a budget ten percent above the current total those three stages can each grow by less than seven units, while stage 14, on two routes, has room for 24 (Table 8.3). Contribution and room order the stages differently, which is the information a capacity decision needs and which no scheduling slack could express.

Table 8.3: Accumulation on the water network under a budget of 528 units at outlet 27 (current total 480). Allowance is headroom divided by path multiplicity.

Node	Contribution	Multiplicity	Allowance
3	63	7	+6.9
6	56	7	+6.9
5	35	7	+6.9
1	30	5	+9.6
11	27	3	+16
14	16	2	+24

Uncertainty then tests how much of the deterministic story survives. With every one of the 32 stage durations widened to ± 20 percent, exhaustive corner enumeration would require 2^{32} propagations, roughly 4.3×10^9 . The domination split required 2,269,328, a factor of 1,892 fewer, with the largest bypass set holding 18 of the 32 durations, and completed in 135 seconds on commodity hardware. The answer changes how the deterministic result must be read. Under full uncertainty no stage is necessarily critical, and exactly nine of the 32 are possibly critical. The single deterministic critical chain dissolves into a nine-candidate set (Figure 8.5), and 23 stages are provably incapable of becoming critical anywhere in the uncertainty box. A Monte Carlo attack on the same bounds, fifty thousand sampled configurations, produced no violation and reached every bound at every node.

8.7.2 The most reliable route

The drone medical delivery network of 782 nodes, 2,789 edges, 33 launch sites and 323 destinations poses a multiplicative question, which route delivers with the highest end-to-end success. The factors are the study’s own component success rates, node priors between 0.75 and 0.95 and per-link probabilities between 0.70 and 0.95, so nothing in the scenario is invented. The best route in the network compounds to a success factor of 0.787, and it is a single direct leg from launch site 380 to destination 393. That brevity is structural rather than accidental. The strongest leg the data permits compounds its link and node factors to 0.9025, so every leg a route adds costs at least 9.8 percent of its success, and no amount of

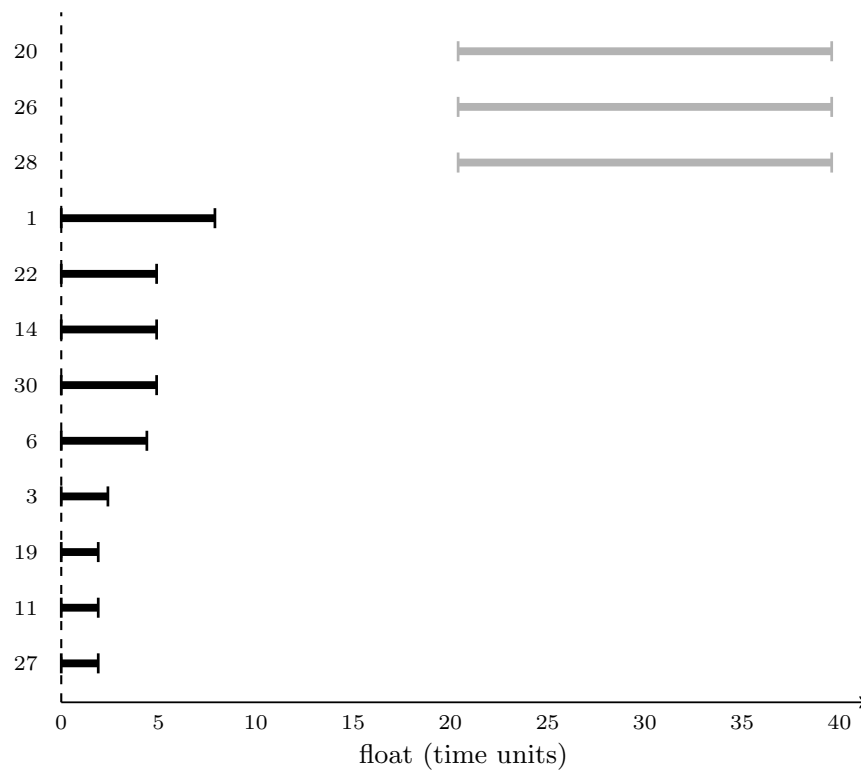


Figure 8.5: Exact float bounds on the water network with every duration at plus or minus 20 percent. Black bars are the nine possibly-critical stages, whose lower bounds reach zero. Grey bars are three of the 23 stages whose floats stay provably positive everywhere in the uncertainty box.

component quality along a longer route can repay that arithmetic. Route brevity is the currency of the multiplicative reading, which nothing in an additive analysis of the same network would reveal.

Table 8.4: The five best-served destinations by single-route success. At the other end of the scale, 104 of the 323 destinations fall below 0.5 on even their best route.

Destination	Best-route success
393	0.787
325	0.771
209	0.747
248	0.742
125	0.734

Read per destination rather than globally, the forward values become a service-level map, and the map is unequal (Figure 8.6). The five best-served destinations sit within seven percent of the optimum (Table 8.4), while the weakest destination in the network can be reached with success 0.108 by its best route, and for 104 of the 323 destinations, roughly one in three, even the best single route succeeds less than half the time. No routing decision alone can serve a third of this network reliably, and where that matters the remedy is structural, stronger components on the thin corridors or genuinely redundant routes. Judging that redundancy is the neighbouring question, because a destination poorly served by its best single route may still be reliably reached when every route counts at once, which is the reachability reading of Chapter 6. The two readings separate route choice from system adequacy, and a delivery planner for this network needs both. Two thousand randomly sampled routes never exceeded the reported optimum and one reached it exactly, and the optimum itself is the product of three published input values, checkable by hand.

8.7.3 The boundary

Dense mesh reconvergence is where the split’s advantage disappears. On a five-by-five grid DAG with all 25 durations interval-valued, the bypass sets hold up to 23 of the 25 durations and the split would need more propagations than one shared exhaustive sweep, which the implementation therefore uses instead. The hardness results guarantee that some regime behaves this way, and it is the same regime in which the probability toolkit’s conditioning sets explode. Reconvergence density is the shared price axis of both chapters, and mapping the boundary is part of the result rather than a qualification of it. Timings are reported only as measured.

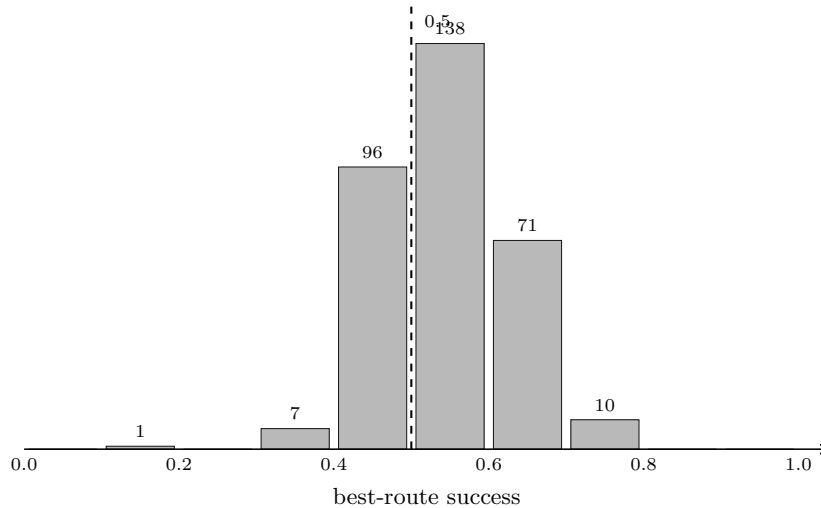


Figure 8.6: Best-route success across the 323 drone destinations. Left of the dashed line at 0.5 lie the 104 destinations that no single route serves reliably.

The water analyses run in 53 microseconds per pass for the additive modes and 7 for accumulation, the drone network in 1.8 milliseconds, and the exhaustive comparisons are quantified by run counts rather than by extrapolated wall-time.

8.8 Genericity

The boundary of the toolkit is drawn by the algebra rather than by the implementation. A new quantity joins by exhibiting its operator pair, and the pair earns its backward pass the way the shipped ones did, by carrying a residual or by being linear. The obligations are laws, verified on sampled values before a mode is accepted, so an extension can never silently acquire a backward pass its algebra does not support. The kernels are generic over the value they propagate, and interval support is a scheme wrapped around crisp runs rather than a property of a value type, which is what keeps its exactness claims provable.

The nearest extensions follow the existing arguments. The shortest-path dual of the domination split mirrors Section 8.6 and awaits only the line-by-line derivation. Interval-valued delays reduce to interval durations by subdividing their edges. Further out, neighbouring nodes have overlapping bypass sets and therefore overlapping corner evaluations, an economy of the kind the hash-consed store of Chapter 5 already practises for diamonds, and the exclusion predicate of that chapter’s identification algorithm has in fact run with a zero-width interval definition during the development of the split, so the decomposition module’s extension surface is known to support value semantics other than probability.

8.9 Summary

The toolkit reads one graph object in two directions. Forwards it computes the extremal value of any quantity that accumulates along dependency paths. Backwards it computes the room each component has before that value moves, wherever the quantity’s algebra defines such a reading, which is the condition every shipped mode satisfies and every future mode must. The classical critical path method is the default instance rather than the design, and the sum family reports sensitivities and allowances because that is what its mathematics defines. Under interval uncertainty the forward quantities stay exact for the price of two crisp runs. The floats are exact through the domination split, whose correctness rests on three short proofs and whose cost grows with the reconvergent width around each node rather than with the number of uncertain inputs. That width stayed small enough on a real network to solve a 32-duration instance beyond the reach of exhaustive enumeration, it does not stay small on dense meshes, and it vanishes exactly on the series–parallel structures where the decomposition module of Chapter 5 would also find nothing to decompose. Every number reported in this chapter is certified against an independent oracle, an exhaustive enumeration or a sampling attack.

Chapter 9

Package Implementation in Julia

Chapter 10

The No-Code Web Interface

Chapter 11

An End-to-End Framework

Application: An Integrated Case Study